



TM252

Functions, function blocks and libraries

Requirements

Training modules	TM210 - Automation Studio TM246 - Structured Text TM251 - Memory organization and IEC programming
Software	Automation Runtime 6.0 Automation Studio 6.1
Hardware	ArSim / X20CP1686X

Table of contents

1 Introduction.....	4
1.1 Learning objectives.....	4
1.2 Symbols and safety notices.....	4
2 Functions.....	5
2.1 Using functions.....	5
2.2 Creating your own functions.....	7
2.3 Functions in the stack.....	13
3 Function blocks.....	17
3.1 Using function blocks.....	17
3.2 Creating your own function blocks.....	19
3.3 Standardized interfaces and defined behavior.....	22
4 Libraries.....	27
4.1 B&R libraries.....	27
4.2 B&R library samples.....	30
4.3 User libraries.....	32
5 mapp Technology.....	43
5.1 Concept.....	43
5.2 Components of a mapp Technology Package.....	44
5.3 mapp framework.....	49
6 Summary.....	50
7 Solutions.....	51

1 Introduction

When creating complex applications, the structure of the source code in the project and efficient use of system memory resources are of particular importance. This is the only way to ensure easy maintenance and allow the project to be reused as a whole or in individual parts.

To structure the source code, the option of using functions and function blocks and grouping them together in libraries is useful.

B&R offers a wide range of standard libraries and a variety of functions to enable fast and efficient programming. mapp Technology is an extension of the library concept that provides ready-made solutions for simple commissioning and programming, and even for complex application scenarios.

If user-specific functions and function blocks are also required, these can be conveniently exported in Automation Studio via user libraries.

1.1 Learning objectives

With selected application samples and exercises, course participants will learn how to use functions, function blocks and libraries correctly.

The following contents are covered:

- Comparing the options of actions, functions and function blocks.
- Using actions, functions and function blocks in your project.
- Understanding the system behavior when calling functions and function blocks.
- Learning about the large number of existing B&R libraries and library examples and applying them according to your requirements.
- Creating your own actions, functions and function blocks.
- Creating function blocks with standardized interfaces.
- Creating user libraries and make them available outside your Automation Studio project.
- Explaining the concept of mapp technology.

1.2 Symbols and safety notices

Unless otherwise specified, the symbol descriptions and safety notices listed in "TM210 - Working with Automation Studio" apply.

2 Functions

If the same instructions are required in several places during program development, these instructions can be integrated in a function. Instead of a list of instructions, it is only necessary for the function to be called at relevant points in the source code.

This increases the clarity of the source code, saves system resources and makes parts of the project reusable for other applications.

The instructions "behind" the function name are executed within one cycle. Processing all instructions must be completed before the end of the cycle time; otherwise, this will result in a cycle time violation on the system.

2.1 Using functions

In the following section, how a function is used is explained using `brsmemset()` as an example.

A function is always identified by its **name**. In this case, the name of the function is `brsmemset`. According to the Application Design Guidelines (TM233), the name of the function indicates the task that function fulfills.

A function's **parameters** provide the information the function requires in order to execute the specified instructions. Function parameters are placed in round brackets after the function name and cannot be changed in terms of number, sequence and data type. The individual parameters are separated from each other by commas.

The **return value** of a function is transferred back to the calling program when the function is completed. This can be either the result of a calculation or status information. The return value of a function also has a defined data type. A function always has exactly one return value.

The following information can be found in the documentation for the `brsmemset()` function:

- The task to be carried out by the function is placing a defined value in a specific memory area.
- The function has three parameters of type UDINT, USINT and UDINT.
- The return value has data type UDINT.

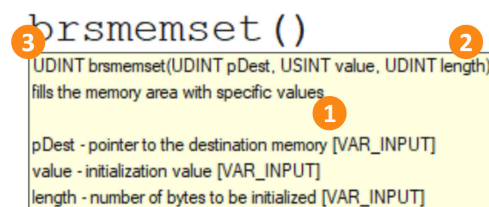


Figure 1: Detailed information about `brsmemset()` via the SmartEdit function in Automation Studio

Before using the function in a program, the parameters must be examined in more detail. To call a function correctly, the order of the parameters must be taken into account in addition to the data types:

UDINT	<code>brsmemset (</code>	<code>UDINT pDest ,</code>	<code>USINT value ,</code>	<code>UDINT length)</code>
Data type of the return value	Name of the function	The first parameter is of type UDINT and is the memory area for writing.	The second parameter is of type USINT and is the value that is to be written to the memory area.	The third parameter is of type UDINT and specifies the number of bytes in memory starting from address <code>pDest</code> that should be written to with <code>value</code> .



Initialize a memory area with the value 252

Declarations

VAR

```
Memory : ARRAY[0..99] OF BYTE;
Value : USINT := 252;
NextAdr : UDINT;
```

END_VAR

The memory area that is to be initialized is named `Memory`.

A variable named `NextAdr` of type `UDINT` is declared in order to be able to collect the return value of the function in the program.

Function call

```
NextAdr := brsmemset(ADR(Memory), Value, 50);
```

According to the declaration for `brsmemset()`, the start address of the memory area to be written to must be transferred as the first parameter when the function is called. This address is obtained using the function `ADR()`¹.

The second parameter is the value to be written to the memory area. In the example, the variable `Value` was declared for this purpose.

Instead of a variable, a value can also be transferred directly as a parameter. This is shown by the third parameter in the example, which has the value 50. This means that the value 252 is written to 50 consecutive memory cells in the memory.

In the example, assigning the return value of the function to the `NextAdr` variable makes it possible to continue writing to the memory area after the first 50 values have been written:

```
Value := 12;
NextAdr := brsmemset(NextAdr, Value, 25);
```

This is used to write the (new) value 12 to the next 25 memory cells.

The return value of a function can be stored by the calling program in a program variable, as shown in the example. The return value can also be evaluated directly, however, for example in an IF query. If the return value is not required at all, the function call can also be made without assigning the return value.



If parameters are transferred with incorrect data types, this results in an error message during the compilation process.

The names of the parameters are not of importance when calling a function.



[Programming \ Functions and functions and function blocks \ Functions](#)

Task: Stopwatch

The `clock_ms()` function from library `astime` should be used to implement a stopwatch. The following requirements must be met:

- 1) A Boolean trigger variable `ResetTime` resets the time count of the stopwatch to 0.
- 2) The stopwatch starts counting the time automatically without a start signal.
- 3) The current count value of the stopwatch should be stored in the variable `TimeDifference` of type `TIME`.

Create Watch window entries to check the functionality of your stopwatch.

Information: To use a function, the library² that contains the function must exist in the project.

¹ Strictly speaking, the return value of function `ADR()` is the address of the variable `Memory`, which is transferred as a parameter.

² A library must also be available on the target system. Libraries are automatically added to `CPU.sw` by the compiler, but can also be added manually to the `.sw` file.

2.2 Creating your own functions

2.2.1 Transferring parameters to a function

The parameters that need to be transferred to a function, which were mentioned previously, can be divided into two categories that result in different behaviors.

A distinction is made between the following types of **parameter transfer**:

As a copy of a value ("by value")

- When the function is called, it is not the variable itself that is transferred to the function, but only a copy of the value that the variable has at the time the function is called.
- A variable transferred "by value" always remains unchanged when the function is called because the function does not have write access to this variable.
- In the declaration file, these are all variables that can be found under **VAR_INPUT**.

As a pointer ("by reference")

- Instead of a fixed value, the address of a variable is transferred to the function when it is called.
- The function therefore has write access to the memory cell. This means that the function can directly change the value of a variable transferred "by reference".
- In the declaration file, these are variables of category **VAR_IN_OUT**.



When declaring a function, it is necessary to specify whether a parameter is transferred as a copy of a value ("by value") or as a pointer ("by reference").



STRING, WSTRING, fields and structures can only be transferred as pointers, which means that changes are always made to the original object!

2.2.2 Declaring and defining functions

If you want to create your own functions for special tasks, you will need both a function declaration and a function definition.

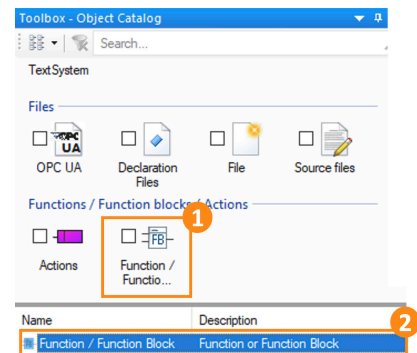
The name, the data type and number of parameters and the data type of the return value are defined in the **function declaration**. The function is made known to the compiler with the declaration, similar to the declaration of a variable.

The actual instructions that are to be executed when the function is called are stored in the **function definition**. The function definition is also referred to as the implementation of the function.

In Automation Studio, the declaration of one (or more) functions takes place in a ".fun" file. The function definition(s) are saved in source code files with the extension of the programming language used for implementation. In the case of structured text, this is ".st".

As both function declarations and function definitions are software components, they are added from the Toolbox into the Logical View.

- 1) Filtering to "Function / Function block" makes the search easier.
- 2) When "Function / Function block" is added, a window opens to assist entering all the information required for the function.



The following information about the function can now be entered in the dialog box:

Figure 2: Dialog box for creating a function

- 1) Name of the function or function block that is being created.
- 2) Whether a function or function block should be created.
- 3) Name of the file for implementing the function. By default, this is the name of the block itself with the file extension corresponding to the programming language.
- 4) Selection for the desired programming language.
- 5) Specification of the data type for the return value of the function.
- 6) In the next step, another window opens where the function's parameters can be specified using the corresponding data types.

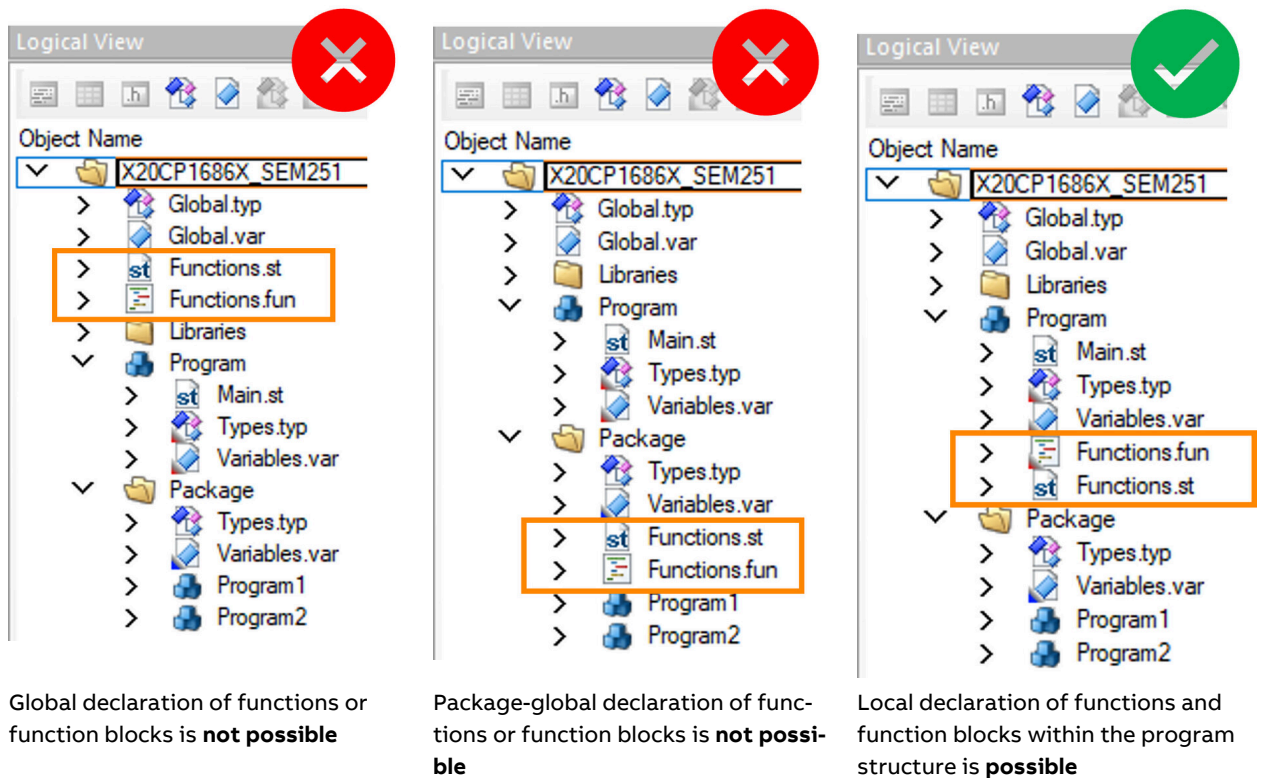
Once the dialog box is closed, both the declaration file (.fun file) and the definition file (source file) for the function are available in the project with the information entered.

All function details entered in this dialog box can then be changed or adapted in the .fun file. Care must be taken to ensure consistency with the "source file", i.e. the file where implementation takes place.

If functions are created manually by adding the object "New file" from the Toolbox, it is necessary to manually assign the extension ".fun" to the file for the declaration. The same applies to the source file for implementing the function.

2.2.3 Scope of functions

Individual functions cannot be created globally for a project in Automation Studio, but only locally for a program; these are referred to as program-local blocks.



If you want to make functions globally available, you have to integrate them into a library, and then they are available for all programs in the project.

2.2.4 Function declaration in Automation Studio

After adding the two files for declaration (".fun" file) and definition (".st" file), these files can now be used to create the application-specific function. This process always starts with the function declaration in the ".fun" file.

The function declaration is started with the keyword `FUNCTION`.

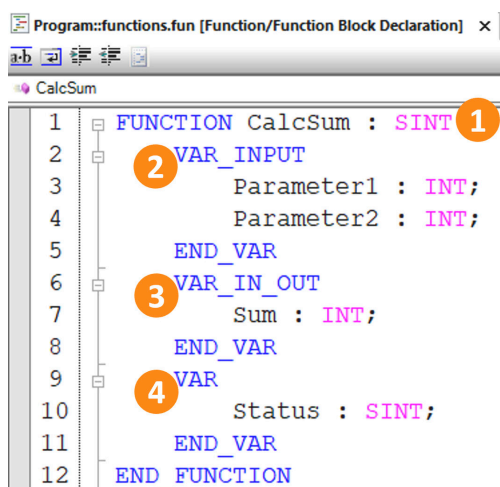


Figure 3: Declaring the `CalcSum()` function in the text editor

The function declaration is completed with the keyword `END_FUNCTION`.

- 1) `CalcSum` is the name of the function that is being declared. `SINT` is the data type of the function's return value.
- 2) The keyword `VAR_INPUT` is used to declare all function parameters that are transferred "by value".
- 3) `VAR_IN_OUT` is used to transfer parameters "by reference". It is not necessary to reference the function call (as in C, for example), this is done automatically in ST. Within the implementation of the function, dereferencing also takes place automatically.
- 4) All variables declared after the keyword `VAR` are only recognized and usable within the function. They are considered "local" variables of the function itself.

A function can also be declared in the table view for the ".fun" file. Here, you can clearly see the significance of VAR_INPUT, VAR_IN_OUT and VAR in the declaration of the function variables.

Name	Type	& Reference	Scope
CalcSum	SINT	☐	VAR_IN_OUT
Parameter1	INT	☐	VAR_INPUT
Parameter2	INT	☐	VAR_INPUT
Sum	INT	☒	VAR_IN_OUT
Status	SINT	☐	VAR

Figure 4: Declaring the CalcSum () function in the table editor

- 1) Name of the function and data type of the return value
- 2) VAR_INPUT: Values go "into" the function. They can be changed within the function, but nothing changes outside the function. This is due to the fact that the only thing taking place is a number being supplied to the function.
- 3) VAR_IN_OUT: Something goes "into" the function, and something also comes "out" again. Transferring "by reference" means that value changes to the variable are also visible outside the function and are therefore "retained" even after the function has been completed.
- 4) VAR: These variables have no bearing on the "outside world", which is illustrated by the image without arrows.

Local variables for a function can also be declared as constants. These are then only recognized and usable within the function, however.



The interfaces for a function, i.e. the parameters and the return value, must be documented, especially if passed on to others. Otherwise, the user has no information about correct use. Writing comments is therefore recommended.

The interfaces for all implemented B&R functions are [documented in Automation Help](#).

2.2.5 Function definition in Automation Studio

After declaring a function, implementation can take place, i.e. creating the source code with the instructions that are to be executed each time the function is called. This is done in a source code file with the extension corresponding to the programming language.



In visual programming languages, each block (function or function block) must be implemented in a separate source file; in text-based programming languages, it is possible to combine several or all declared functions in one source file.

A function definition also begins with the keyword FUNCTION, followed by the name of the function for which instructions should be defined. Possible implementation of the CalcSum () function could look like this:

```

1 FUNCTION CalcSum
2     Sum := GetSum(Parameter1, Parameter2);
3     IF Sum > 0 THEN
4         Status := 1;
5     ELSIF Sum < 0 THEN
6         Status := -1;
7     ELSE
8         Status := 0;
9     END_IF
10    CalcSum := Status;
11 END_FUNCTION

```

All variables previously declared for the function can now be used in the function definition.

The return value of the function is defined in the last line of the implementation as
Function name := Return value;
.

The function definition must be completed with
END_FUNCTION.

Figure 5: Implementation (definition) of the CalcSum () function

Sum is transferred to the function as a pointer, so the calculated value of the sum is written directly to the variable in the program where it was called. The return value of the function indicates the sign of the sum calculated by the function.

If a function must be terminated prematurely because invalid parameters were transferred when it was called, for example, this can be achieved using the keyword `RETURN`. Press `RETURN` to abort the processing of the function, return to the called instance and continue executing the command there.



[Programming \ Programs \ Structured Text \(ST\) \ RETURN statement](#)

2.2.6 Using the CalcSum() function

Once the function has been declared and defined, it can now be used in the cyclical section of the program. This is referred to as a function call.

If comments have been entered in the function declaration, Automation Studio can display them in the program when the function is used.

This shows the user that Sum is transferred as a pointer `[VAR_IN_OUT]`. The value of Sum can therefore change as a result of the function call.

```
CalcSum()
SINT CalcSum(INT Parameter1, INT Parameter2, INT Sum)
adds two values, returns whether result was positive, negative or zero

Parameter1 - first summand [VAR_INPUT]
Parameter2 - second summand [VAR_INPUT]
Sum - result of calculation [VAR_IN_OUT]
```

Figure 6: Detailed information about `CalcSum()` via the SmartEdit function

Variable declarations

```
VAR
  Result : INT;
  Number1 : INT := 5;
  Number2 : INT := 7;
  Status : SINT;
END_VAR
```

Possible function calls

```
Status := CalcSum(Number1, Number2, Result);
CalcSum(10, -20, Result);
```

To call function `CalcSum()` in the cyclic section of the program without errors, the variable where the result of the calculation is saved must be declared as a minimum requirement.

The first function call will save the value 12 in `Result` and the value 1 in `Status`.

The second function call will overwrite the existing value of `Result` with -10, the value of `Status` remains unchanged because the return value of the function is not assigned again as `Status`.

The names of the variables that are transferred to the function can – but do not have to – match the names of the declared parameters.



When implementing a function, other functions already declared in the project can be used at any time. This means that existing functions can often be very easily expanded to include an additional aspect. If the existing function is part of a library, it must first be integrated in the project; otherwise, an error message indicating an unknown function will be displayed during compilation.

Task: Creating the WritePizzaString() function

A function `WritePizzaString()` should be created that saves three (freely selectable) values of a `PizzaType` variable in a formatted string, e.g:

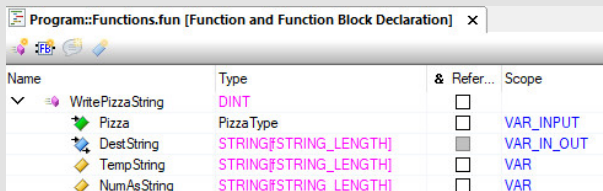
```
Price:=8.5;Size:=32;LeaveCheese:=TRUE;
```

The string should not contain any spaces, the separator of the elements is a ";" and each value should be preceded by the corresponding element name with a ":=".

The target string should be a `VAR_IN_OUT` parameter. The return value of the function should be of type `DINT` and used for status or error messages.

Procedure:

- 1) Add a .fun file to a program in your project and declare the function with the parameters and return value of type `DINT`.



Name	Type	& Refer...	Scope
WritePizzaString	DINT	☐	
Pizza	PizzaType	☐	VAR_INPUT
DestString	STRING#STRING_LENGTH	☒	VAR_IN_OUT
TempString	STRING#STRING_LENGTH	☐	VAR
NumAsString	STRING#STRING_LENGTH	☐	VAR

Figure 7: Declaration of WritePizzaString()

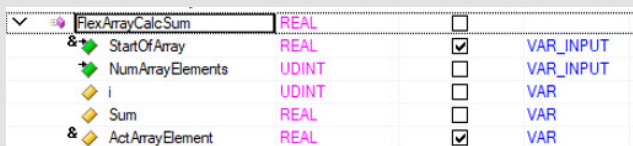
- 2) Add a source code file and define the function.
- 3) Call up the function in the program and test the functionality
- 4) **Addition:** Perform a length check in your function before writing `DestString`. If the target string is too short, the function should be terminated prematurely and the string returned empty. In this case, the function should also return a defined error number via the return value.

Task: Creating the FlexArrayCalcSum() function

A function should be created that calculates the sum of all elements of an array. The size of the array should be flexible. The number of elements must therefore also be transferred to the function. The return value is the sum of all elements.

Procedure:

- 1) Declare the function accordingly with the necessary parameters in a .fun file.



Name	Type	& Refer...	Scope
FlexArrayCalcSum	REAL	☐	
StartOfArray	REAL	☒	VAR_INPUT
NumArrayElements	UDINT	☐	VAR_INPUT
i	UDINT	☐	VAR
Sum	REAL	☐	VAR
ActArrayElement	REAL	☒	VAR

Figure 8: FlexArrayCalcSum() declaration

- 2) Create the function definition in a source code file.

Note: With

```
ActArrayElement ACCESS (ADR(StartOfArray) + Offset);
```

you can reach all elements of the array within a loop through a variable offset.

- 3) Call your function in the program and test the function by manually changing individual values of the array in the Watch window and observing the return value of the function.

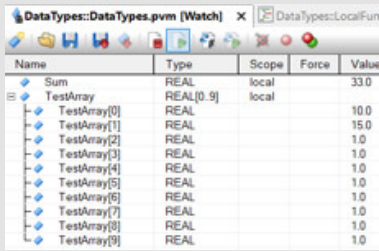


Figure 9: Calculation of the sum with FlexArrayCalcSum()

2.3 Functions in the stack

When a function is called, a so-called "stack frame" is created in the stack (memory area in the DRAM) for the function, which contains the variables of the function³. The stack frame of a function only exists as long as the function is being executed, i.e. as long as the instructions of the function are being processed. The following example shows what happens when a function is called and completed. The gray memory area represents the stack frame of the calling program, the blue memory area represents the stack frame of the CalcSum() function as declared in the previous section.

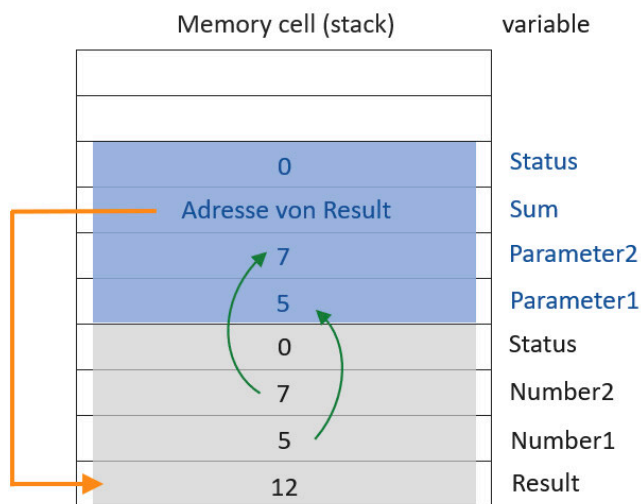
Memory cell (stack)	variable
0	Status
7	Number2
5	Number1
0	Result

Before calling the function

The stack only contains the variables of the running program.

For better readability, the image only uses one "memory cell" for each variable. The actual number of memory cells used per variable depends, of course, on the data type of the variable.

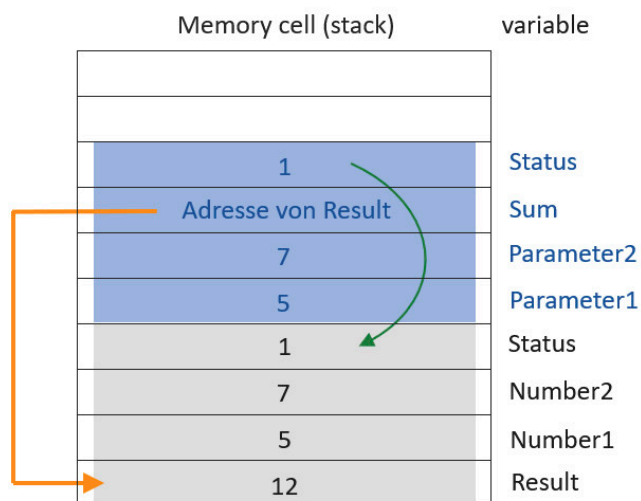
³ The stack frame actually contains even more information than just the variables of a function, such as the return address to the calling function but this is not relevant here.



Calling the CalcSum function

A separate stack frame (light blue) is created for the CalcSum function, which contains the function's variables.

The **values** that are transferred as parameters are now copied into the variables (green arrows). For variables that are transferred "by reference", the address of the referenced variable is delivered to the function. This effectively creates a **pointer** from the function that points to the transferred variable (orange arrow). In the example, the value of the calculated sum is therefore already written to `Result` during the function's runtime!

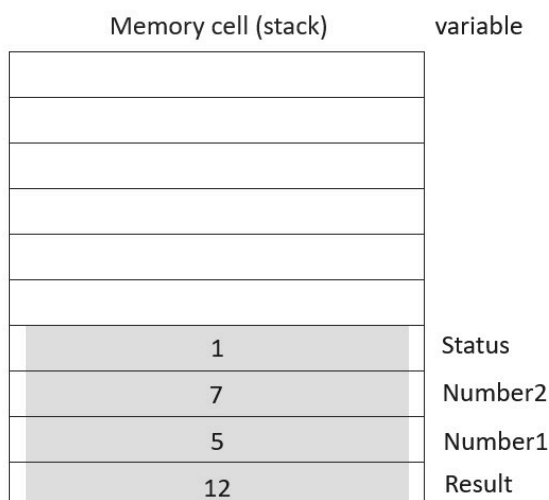


Terminating the CalcSum function

The function's return value is given back to the calling program.

In the example (see above), the return value is stored in the `Status` variable of the program that calls the function.

Here, you can also see that the names of the function's variables and the names of the calling program's variables can be identical, but always use different memory cells in the memory.



After ending the function

When the function is completed, the memory area with the function's "local" variables is freed up again. The stack frame is "closed". It is then available to the system again.

This also explains why the values of the parameters in the function can be changed, but this is "forgotten" after the function is completed. The corresponding memory cells are no longer occupied.

In summary, the following can be said about functions:

- The stack frame of a function is created anew each time the function is called and deleted again after it has been processed.
- All function variable values are lost after a function is completed.
- The return value and information written using a pointer are retained in the calling program.



A function is always processed within one cycle. If the processing time of a function is longer than the cycle time of the calling program, a cycle time violation occurs. This is particularly important when using recursive functions!

Recursive functions

When a function calls itself, this is referred to as a recursive function. This is used in programming if it makes it possible to reduce a task to a smaller instance of the same task.

The recursion results in a nesting of the function calls. This allows for clear source code, but often makes longer run-times and higher memory requirements necessary.

In some cases, iteration can be an alternative to recursion. Here, a series of identical instructions is used instead of nesting. With this type of programming, the source code is often more difficult to read, but the runtime and memory requirements of the function remain lower.

Example: Fibonacci sequence

A comparison of recursive and iterative implementation of the same task will be shown using the calculation of Fibonacci numbers as an example.

Fibonacci sequence 0 1 1 2 3 5 8 13 21 34 55 89 144 ...

Obtained by adding the two previous numbers for each new number in the sequence. From a programming point of view, the calculation of any number *n* in the sequence can be solved as follows:

	Recursive calculation	Iterative calculation
Function declaration	<pre>FUNCTION FibRecur : INT VAR_INPUT n : INT; END_VAR END_FUNCTION</pre>	<pre>FUNCTION FibIter: INT VAR_INPUT n : INT; END_VAR VAR a : INT := 0; b : INT := 1; i : INT; END_FUNCTION</pre>
Function definition	<pre>FUNCTION FibRecur IF n = 0 THEN FibRecur := 0; ELSIF n = 1 THEN FibRecur := 1; ELSE FibRecur := FibRecur(n-1) + FibRecur(n-2); END_FUNCTION</pre>	<pre>FUNCTION FibIter FOR i:=0 TO n-1 DO b := b + a; a := b - a; END_FOR FibIter := a; END_FUNCTION</pre>

At first glance, the definition of the recursive variant of the function appears clearer and the function only requires a single function variable compared to its iterative variant. However, a closer look reveals that `FibRecur (n)` has to calculate identical values multiple times for large *n* values:

3 Function blocks

When using functions, they can have a maximum of one return value. Furthermore, it is not possible to execute actions that require longer than one cycle. This is a problem when using timers, for example.

Function blocks are an extension to the concept of functions. Since they are referred to as function blocks in English, the term "function block" is also sometimes used in German. The abbreviation "FB" is also frequently used.

3.1 Using function blocks

Both a function and a function block can have multiple parameters, but a function block can also have multiple return values. A function block is thus divided into inputs (parameters) and outputs (return values).

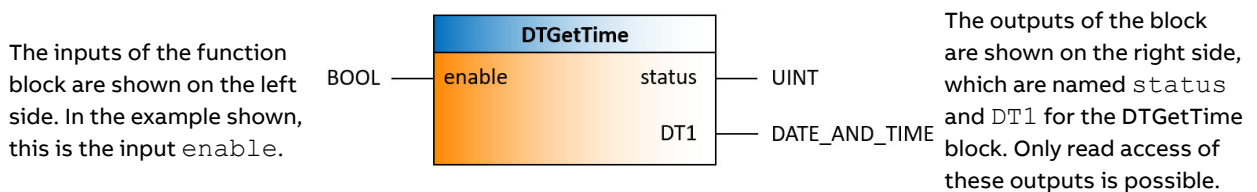


Figure 11: Visual representation of the timer function block `DTGetTime`

Variables that are attached to the inputs and outputs of the function block must correspond to the declared data type respectively. If implicit type conversion is not possible, however, error messages will occur during compilation.

To be able to use a function block, an "instance variable" for the function block type must be created.

This variable is similar to a structure variable. The individual elements of the structure are the inputs and outputs of the function block. The input elements have write access. This means that the values of the input parameters can be set via software. The values of the output parameters can only be read. If you try to initiate write access to a function block output, the compiler returns an error message.

The instance variable exists in the calling program as usual as a local variable. Changes that are made to the inputs and outputs of the instance variable when the block is called are therefore retained even after the block has been processed. It is also possible that output values change after the function block is called again with the same call parameters depending on the "history" of the instance variable⁴. Function blocks are also said to have a "static memory".

As implemented, the function block itself only accesses the "input elements" in read-only mode; the "output elements" are written by the function block code.



Function blocks are usually components of libraries. If a function block is to be used, the associated library must also be imported into the project.

⁴ For example, outputs of timer function blocks switch depending on the elapsed time, although nothing has changed on the inputs.

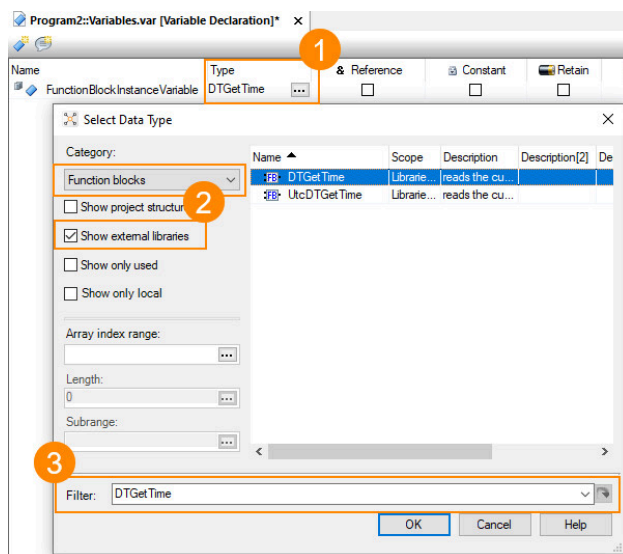


Figure 12: Dialog box for declaring a function block variable

Like all other variables, function blocks instance variables are also declared in the .var file.

- 1) Instead of a basic data type, the corresponding function block must be specified here.
- 2) Function blocks from libraries that have not yet been imported into the project can also be displayed and selected. The associated library is then automatically imported into the project when the declaration dialog box is closed.
- 3) A filter option can be used to restrict the function blocks displayed according to their names.

Once the instance variable has been created, the individual inputs and outputs of the function block can be accessed in the source code using a dot operator, just as with structure variables.



Using a timer function block

The STANDARD library contains various timer function blocks that can be used to implement switch-on or switch-off delays and pulse generators.

In the example, a switch-on delay of 20 s is implemented using the TON function block.

Declaration

```
VAR
    Switch : BOOL;
    Lamp : BOOL;
    Timer : TON;
END_VAR
```

Lamp should be switched on 20 seconds after Switch is switched on.

The instance variable for the function block is called Timer.

Program code

```
Timer.IN := Switch;
Timer.PT := T#20s;
Timer(); // Call of function block
Lamp := Timer.Q;
```

After specifying the function block inputs, the function block must be called. Only then is the code that is stored for the block actually executed.

Alternatively, the function block inputs can also be specified directly when calling the function block:
 Timer(IN:=Switch, PT:=T#20s);



It is important that calling the function block in the program is not forgotten!



Not all inputs need to be specified to call a function block. If the values on the inputs of a function block are changed, the block must be called again. A function block is only permitted to be called once per cycle.



[Programming \ Functions and function blocks \ Function block](#)

Task: After-work lamp with DTGetTime

A lamp (digital output) should light up from a specified time until midnight. The time should be read using function block `DTGetTime()`.

- 1) Create three variables. One of type `BOOL` for the lamp, one of type `TIME_OF_DAY` for the closing time (e.g. 5:00 PM) and an instance variable of function block `DTGetTime`.

Name	Type	Value
 TimeToGoHome	TIME_OF_DAY	TOD#17:00:00
 doLamp	BOOL	
 GetDate Time	DTGetTime	

- 2) Switch the lamp on as soon as the current time exceeds the defined closing time.

Note: Output `DT1` of the function block and variable `TimeToGoHome` have different data types!

3.2 Creating your own function blocks

For more complex applications, creating your own functions and function blocks is an important part of structured programming.

The procedure for declaring and defining your own function blocks is the same as for creating user-defined functions. The declaration of the function block takes place in a `.fun` file, the block is implemented in a source code file for the corresponding programming language.

In the following, a function block with the name `FlexArrayInfo` should be created that returns the sum of all elements, the average value of all elements and the value of the largest and smallest element for an array of type `REAL` with any number of elements.

To be able to use arrays of different sizes, the start address of the array `pArrayIn` and its size (number of array elements) `Size` are transferred to the function block.

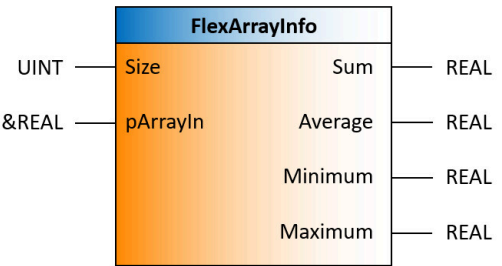
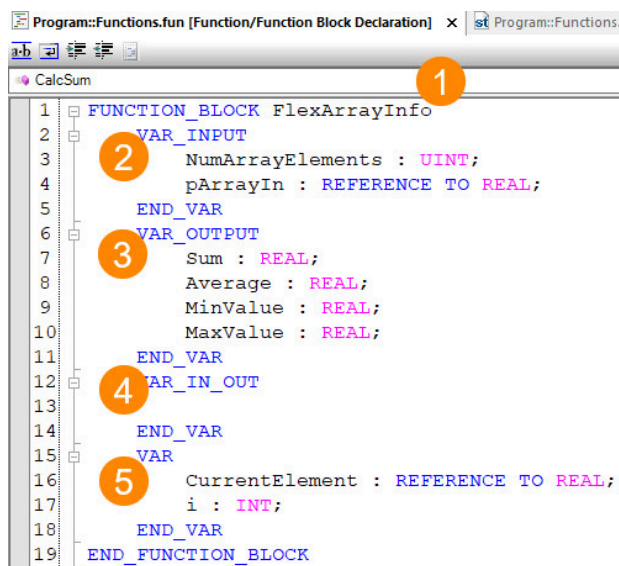


Figure 13: Visual representation of the `FlexArrayInfo` function block

Declaration of function block `FlexArrayInfo`



```

1 FUNCTION_BLOCK FlexArrayInfo
2   VAR_INPUT
3       NumArrayElements : UINT;
4       pArrayIn : REFERENCE TO REAL;
5   END_VAR
6   VAR_OUTPUT
7       Sum : REAL;
8       Average : REAL;
9       MinValue : REAL;
10      MaxValue : REAL;
11  END_VAR
12  VAR_IN_OUT
13  END_VAR
14  VAR
15  END_VAR
16  CurrentElement : REFERENCE TO REAL;
17  i : INT;
18  END_VAR
19 END_FUNCTION_BLOCK

```

Figure 14: Declaration of function block FlexArrayInfo in the .fun file

- 1) The declaration of a function block begins with the keyword `FUNCTION_BLOCK` followed by the name of the block (here: `FlexArrayInfo`)
- 2) All function block inputs are declared using `VAR_INPUT`.
- 3) All function block outputs are declared using `VAR_OUTPUT`.
- 4) With `VAR_IN_OUT`, references (pointers) can be transferred (as with functions) so the function block has write access to a variable outside its own data structures in memory.
- 5) All local variables that are only known within the function block are declared using `VAR`.

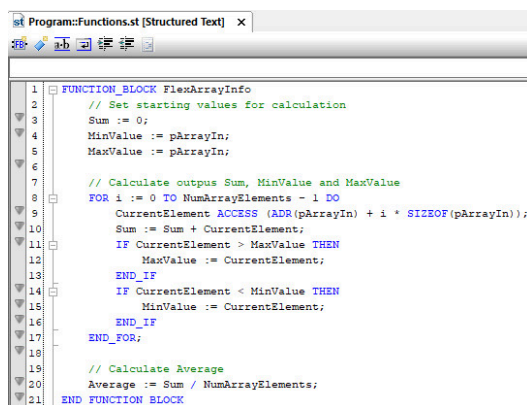
Declaration is completed with the keyword `END_FUNCTION_BLOCK`.



The value of the internal local variable is retained over multiple cycles and therefore different function block calls. They are the "static memory" of the function block and can lead to different values being returned on the outputs even if the input parameters are identical.

Implementation of function block FlexArrayInfo

Once the function block has been declared, it can be implemented (defined) in a source code file. When using visual programming languages, each implementation requires its own file. For text-based programming languages, we recommend using a separate file to implement each function block.



```

1 FUNCTION_BLOCK FlexArrayInfo
2   // Set starting values for calculation
3   Sum := 0;
4   MinValue := pArrayIn;
5   MaxValue := pArrayIn;
6
7   // Calculate output Sum, MinValue and MaxValue
8   FOR i := 0 TO NumArrayElements - 1 DO
9       CurrentElement ACCESS (ADR(pArrayIn) + i * SIZEOF(pArrayIn));
10      Sum := Sum + CurrentElement;
11      IF CurrentElement > MaxValue THEN
12          MaxValue := CurrentElement;
13      END_IF
14      IF CurrentElement < MinValue THEN
15          MinValue := CurrentElement;
16      END_IF
17   END_FOR;
18
19   // Calculate Average
20   Average := Sum / NumArrayElements;
21 END_FUNCTION_BLOCK

```

Figure 15: Implementation of function block FlexArrayInfo

The implementation of a function block begins with the keyword `FUNCTION_BLOCK` followed by the name of the block (here: `FlexArrayInfo`).

In order to be able to calculate the required information (sum, average, minimum and maximum) for arrays of different sizes using the `FlexArrayInfo` function block, only the start address of array `pArrayIn` and its `Size` can be transferred when the block is called.

Actual access to the field elements then takes place via a local pointer (reference variable) `CurrentElement`, which reads over the field in a loop.

The definition ends with the keyword `END_FUNCTION_BLOCK`.

Using the FlexArrayInfo function block

After successful declaration and definition, the block can now be used. As described in the previous section, an instance variable of type `ArrayInfo` must be created for this purpose. One possible area of use for the block is as follows:

Declarations of the variables

```

VAR
  ArrayNumbers : ARRAY[0..9] OF REAL;
  ArrayCalculate : FlexArrayInfo;
  Average : REAL;
END_VAR

```

Source code (cyclic section)

```

// Set input values
ArrayCalculate.Size := 10;
ArrayCalculate.pArrayIn := ADR(ArrayNumbers);
// Call FUB
ArrayCalculate();
// Read output value(s)
Average := ArrayCalculate.Average;

```

In order to be able to test the function block, `ArrayNumbers` is used with ten elements. The instance variable for the function block is named `ArrayCalculate`.

In the cyclic section, values are first assigned to the function block inputs. Instead of a fixed assignment of the number 10, it is also possible to use `SIZEOF(ArrayNumbers) / SIZEOF(ArrayNumbers[0])` making this independent of the array declaration. The calculated average value is saved in the variable `Average`.

Unassigned outputs (such as `Sum`, `MinValue` and `MaxValue` in the example) can also be viewed in the Watch window.



When using `VAR_IN_OUT` variables, please note that a variable can only be directly assigned to this function block input when the function block is called in round brackets.

```
ArrayCalculate(InOutVariable := ProgramVariable);
```

Access using the `.` operator is not possible here and results in an error message from the compiler.

Also, a `VAR_IN_OUT` parameter **must** be assigned a value (i.e. a variable assignment).

Function blocks in memory

The memory for the function block corresponds to the size of the instance variable. Because this is basically something like a structure variable, the same rules apply here with regard to memory requirements and any pad bytes that may occur.

The (structure) elements of a function block variable can be found in the memory in the following order:

- VAR_INPUT variables (memory requirement according to data type)
- VAR_OUTPUT variables (memory requirement according to data type)
- VAR_IN_OUT variables (4 bytes per variable because an address is transferred as a UDINT)
- VAR local variables (only internal function block variables) are also part of the structure and require memory according to data type

All function block variables can be viewed in the Watch window, including the local internal variables.

Even if the order of the variable categories is reversed in the declaration file, the elements of the function block variable can still be found in the memory in the sort order specified above. The pad bytes can occur between data types with different memory requirements.

Exercise: Creating a GetPizzaValues() function block

Function block `FlexArrayInfo()` should extend the calculation of the sum of all array elements of function `FlexArrayCalcSum()` and output the following values at the outputs of the block:

- Sum
- Mean value
- Minimum value
- Maximum value

Procedure:

- 1) Declare a function block named `FlexArrayInfo` and the necessary parameters in a `.fun` file.
- 2) Implement the function block in a source code file. You can use the existing function `FlexArrayCalcSum()` to calculate the sum.

In an empty array (invalid start address or 0 elements), all outputs should output the value 0 and no calculation should be performed.
- 3) Create an instance variable for your function block, assign the inputs in the cyclical part of the program and call your function block. Then check the outputs of the function block in the Watch window.

3.3 Standardized interfaces and defined behavior

When creating function blocks, standardizing the interfaces makes it easier to document and use the software elements.

Old B&R function blocks

Older B&R function blocks work with an `enable` input and a `status` output. The function block instructions are only executed if the `enable` input of the block is set. Error numbers are returned via the `status` output, which provide information about the current status of the block.

Function blocks from B&R libraries provide a constant for each error number, which can be checked for in the code instead of the number. In addition to the block-specific error numbers, the numbers 0, 65534 and 65535 are used as follows:

(Error) number	Constant	Description
0	ERR_OK	Execution successful

(Error) number	Constant	Description
65534	ERR_FUB_ENABLE_FALSE	Enable input for the block not set
65535	ERR_FUB_BUSY	Execution of function block not yet completed
---	---	Various block-specific error numbers

These (error) numbers can be used to wait for a function block to be executed over multiple cycles.

```
IF FileOpen_0.status = 65535 THEN
    // Further calling FB
ELSIF FileOpen_0.status = 0 THEN
    FileOpen_0.enable := 0;
    // Switch to next operation state
ELSE
    // Some error handling
END_IF
```

The defined constants can also be used instead of the numbers, which often makes the code easier to read:

```
IF FileOpen_0.status = ERR_FUB_BUSY THEN
    // Further calling FB
ELSIF FileOpen_0.status = ERR_OK THEN
    FileOpen_0.enable := 0;
    // Switch to next operation state
ELSE
    // Some error handling
END_IF
```

New B&R function blocks

Newer B&R function blocks have standardized interfaces that are used across manufacturers. This is particularly advantageous when using current and modern technologies such as motion or OPC UA.

New B&R function blocks are either **level controlled** or **edge triggered**.

Level controlled

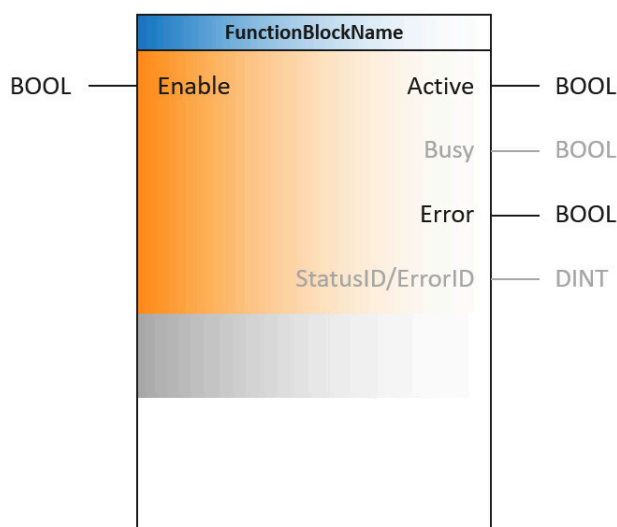


Figure 16: Must-have interfaces for a function block with input **Enable**

Edge triggered

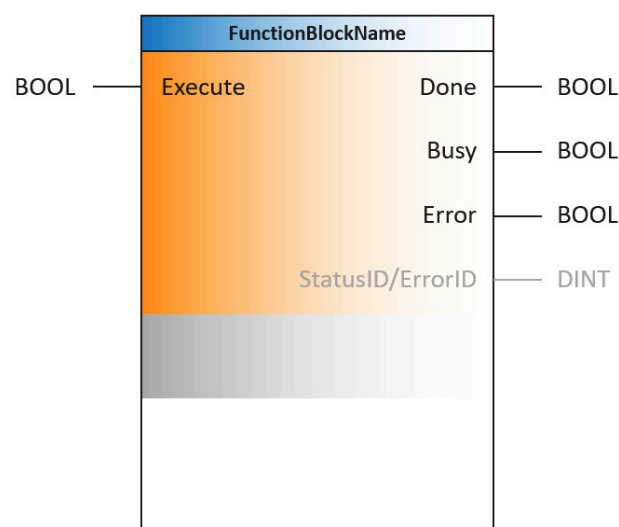


Figure 17: Must-have interfaces for a function block with input **Execute**

The outputs shown purely in gray are possible in the standardized version, but are not mandatory. So they do not have to be present. The **Busy** output indicates whether the function block is still being executed.

- The function block is enabled using an **Enable** input. The function block is only executed if this input is TRUE. If the input is FALSE, the values of all internal variables and the outputs of the function block are reset.
- The **Active** output indicates whether the function block is being executed.
- With function blocks of this type, various actions are often started with additional (command) inputs.
- Edge-controlled function blocks have an **Execute** input. Processing of the function block begins when a rising edge is detected there.
- The successful execution of the block is displayed using the **Done** output.
- When implementing edge-controlled function blocks, you should also consider resetting the **Execute** input.

One use of **Active** in combination with a **Busy** output (level controlled function block) is for function blocks that communicate with hardware where, for example, it is necessary to wait for initialization of the hardware.

All inputs and outputs can be read and written cyclically.

The orange area of a function block shows parameters that are used by multiple function blocks. They are predefined interfaces in terms of behavior.

Inputs and outputs in the gray area are processed and calculated cyclically.

The white area contains additional parameter and command inputs as well as outputs that provide information about execution of the additional commands. A command is only executed after a rising edge at the corresponding input. These inputs and outputs are initiated explicitly.

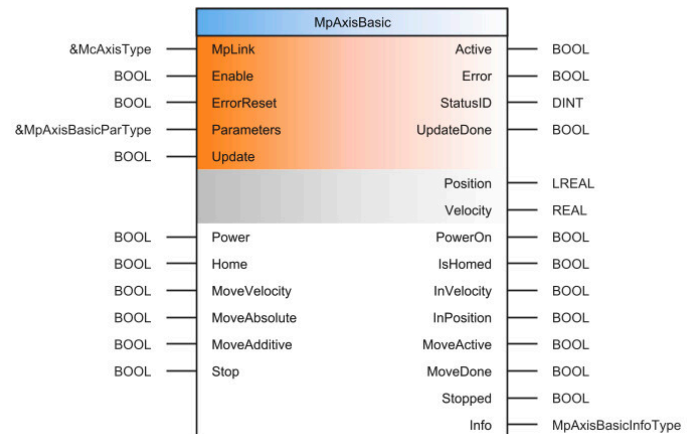


Figure 18: MpAxisBasic as an example of a standardized function block

When using this function, it is important to note that it is often only possible to send a command if certain requirements are met; otherwise, the function block switches to an error state. For example, with function block MpAxisBasic, the command to move the axis can only be executed without errors if the axis is switched on (PowerOn=TRUE) and homed (IsHomed=TRUE). Detailed information can be found in the documentation for the corresponding function blocks.



With a level controlled block, you can also create "multifunction function blocks" that allow you to send various commands via additional command inputs (e.g. Power, Home, MoveVelocity in the white area).

Standardized behavior in the event of an error

Standardized function blocks always have a BOOL type Error output, which is set when an error occurs. Detailed information about the error is then stored in the StatusID or ErrorID output.

- In addition to error numbers, StatusID also contains other numbers with information or warnings
- ErrorID only provides error numbers



A function block can have either an ErrorID output **or** a StatusID output! The two variants are mutually exclusive!



In a library, it is necessary to use either StatusID or ErrorID for all function blocks in the entire library. Mixing both variants (for different function blocks) in the same library is not permitted!

The Error output can be reset in two ways:

- Resetting input Enable / Execute
- Setting input ErrorReset (if available)

Other standards

- All internal variables for a function block should be integrated in a variable with the name Internal and type FunctionBlockNameInternalType.
- If Enable or Execute is reset, all internal variables and the outputs of the function block must also be reset.
- The Done, Busy, Error and CommandAborted outputs are mutually exclusive, which means that only one of them can ever be TRUE.



For a detailed description of all standards that should be followed when creating your own function blocks, see TM232 (B&R Library Design Guidelines).

Exercise: Standardized interfaces for function block FlexArrayInfo()

The interfaces of function block `FlexArrayInfo()` should be standardized.

Procedure:

- 1) Create a function block declaration with standardized interfaces in the .fun file. You need at least one `Execute` input and as well as outputs `Done`, `Busy` and `Error`. Also consider returning error- or status information.
- 2) Implement the additional interfaces of your function block.
- 3) Create and use an instance variable of `FlexArrayInfoStandard()` in the cyclic part of the program and operate input `Execute` of the block in the Watch window.

4 Libraries

Various functions and function blocks that are required for a defined portion of an application or a specific task can be combined in libraries.

Using existing (B&R) libraries makes it easier to create an application because not everything has to be programmed yourself.

Combining custom-made functions, function blocks and data types in a user library helps to structure the project and also allows the programmed functions to be transferred so that they can also be used in other projects.

4.1 B&R libraries

B&R provides libraries for a large number of applications. These libraries are part of the functional scope of Automation Studio and can be imported and used in your projects as required.

4.1.1 Existing libraries

All B&R libraries are documented and described in Automation Help. When importing a library into a project, all data types, constants, functions and function blocks declared in the library are known within the project and can therefore be used from the time of import.

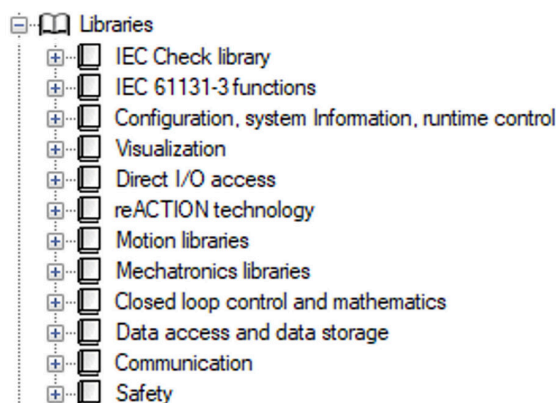


Figure 19: Grouping B&R libraries for application areas

The libraries in Automation Help are divided into different application areas according to their functionality.

This makes it very easy to search for a library even without knowing the exact name.

There is also an alphabetical list of all existing B&R libraries under Programming \ Libraries in Automation Help.

If the name of the library is known, you can access the desired help page via the corresponding link.

There are usually several libraries available for each application area listed in Automation Help.

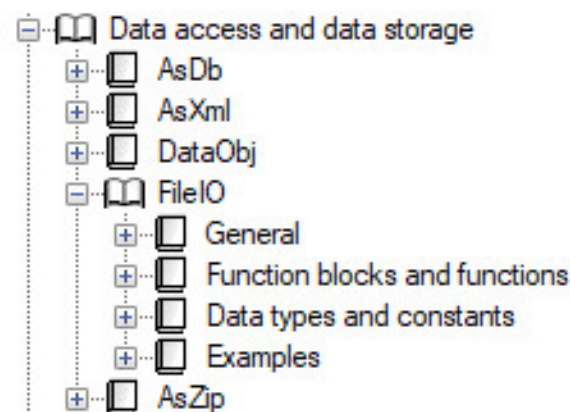


Figure 20: FileIO as an example of a library in the "Data access and data storage" category

The "Data access and data storage" application area includes five B&R libraries:

- AsDb
- AsXml
- DataObj
- FileIO
- AsZip

The documentation includes a general description of functionality and detailed information that allows the function blocks and functions to be used without errors.

This documentation is essential if the function blocks and functions provided by the library are to be used without errors. The data types of interfaces and parameters as well as the type declarations and constants provided in the library are described here. Source code samples for specific use cases are also linked for most libraries.



[Programming \ Libraries](#)

Optional exercise: Options for working with files

- 1) Use Automation Help to find the B&R library that provides functions for working with files.
- 2) Familiarize yourself with the available functions and function blocks and list all those that you need to be able to read from or write to a file without errors.

4.1.2 Using the existing B&R libraries

In an AS project, libraries are managed in the Logical View in a separate "Libraries" folder. Even if libraries are imported in other local project packages, they still have global scope. This means that the functions and function blocks imported with the library are also available outside the package.

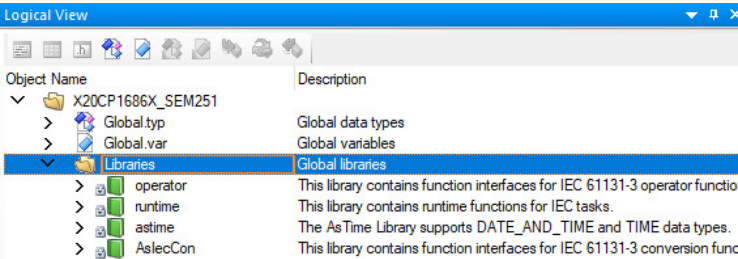


Figure 21: Default libraries in an empty project

When a project is created, the following four libraries are imported by default:

- Operator
- Runtime
- AsTime
- AslecCon

They provide some basic (IEC) functionality. The AslecCon library contains functions for converting data types (explicit type conversions), for example.

In principle, the content of all B&R libraries is structured according to the same scheme, which is briefly explained below using AsTime as an example. The AsTime library provides functions and function blocks that are all related to TIME data types.

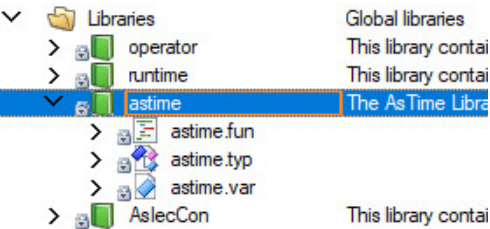


Figure 22: Files contained in a library

Libraries always contain the following files:

- **.fun** file: Declaration file for the functions and function blocks
- **.typ** file: Declaration of all user data types required for the functions and function blocks
- **.var** file: Declaration of constants

The following table provides more details about the content of the three library files:

astime.fun	astime.typ	astime.var
-------------------	-------------------	-------------------

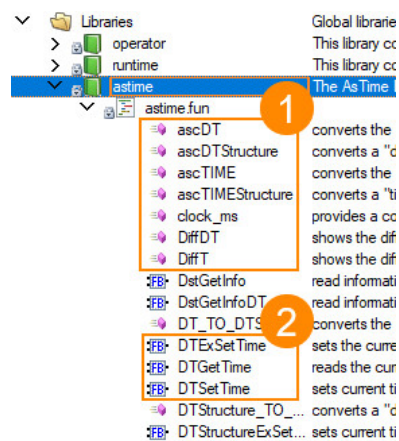


Figure 23: .fun file

When expanding the .fun file in the Logical View, you can see that the AsTime library contains the following elements:

- 1) **Functions**
- 2) **Function blocks**

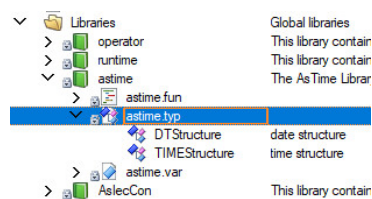


Figure 24: .type file

The structure types are declared in the .type file. In the case of AsTime, these are the two structure types DTStructure and TIMEStructure.

Depending on the library, declared enum types can also be found here.

All enum and structure types declared here can also be used in all other programs in the project once the library is included in the project.

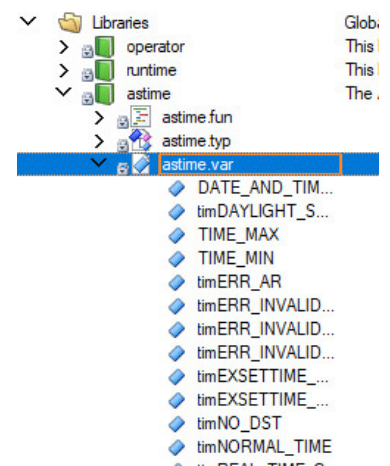


Figure 25: .var file

The .var file for the library contains the constants that are used by the library's functions and function blocks.

All three files can be opened in the editor by double-clicking on them. This allows, for example, the interfaces or parameters for the function blocks and functions to be viewed in detail or the values of the declared constants to be read out.

These files can only be read, however. Making changes to the imported functions, function blocks, data types and constants is therefore not possible.

Importing a B&R library

To import further B&R libraries into an AS project works, complete the following steps:

- 1) In the Logical View, select the package (libraries) where the library should be imported.
- 2) Optional: Use the "Library" filter in the Toolbox to restrict the search results.
- 3) Double-click or drag and drop on "B&R libraries" to open the library wizard.

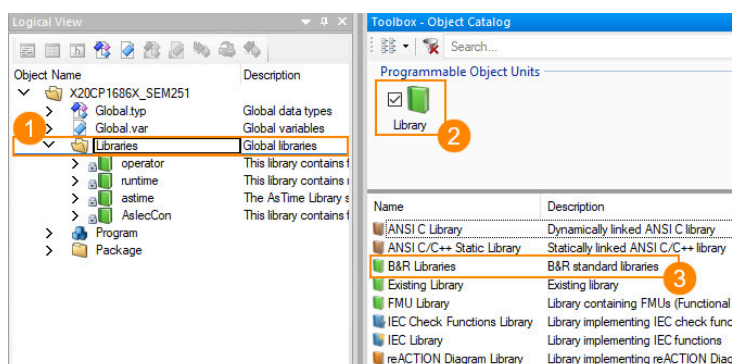


Figure 26: Steps for importing a library

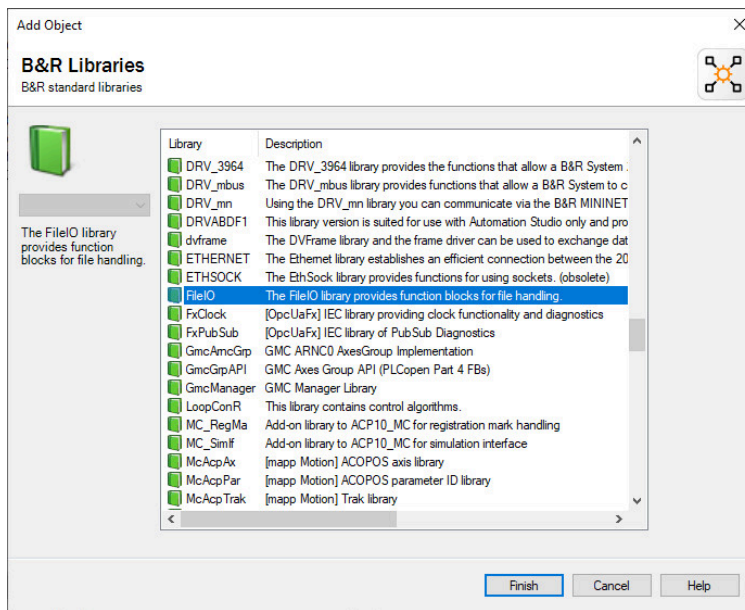


Figure 27: Wizard for importing B&R libraries

A wizard opens where one or more required libraries can be imported into the project.

In the example, the FileIO library is imported into the project, which provides function blocks for working with files on a file system.

When importing a library, it is possible that other libraries are also imported in addition to the actively selected library. This indicates that the library accesses functions and function blocks from the other imported libraries. These are referred to as "dependencies" (4.3.2 "Exporting and transferring" on page 36).

When importing the lecCheck library, the two libraries sys_lib and AsBrStr are automatically imported because the lecCheck function blocks require them.

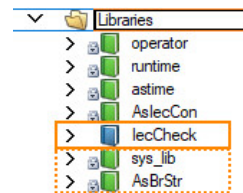


Figure 28: lecCheck library dependencies



For a clear project structure, we recommend including all libraries in the "Libraries" package. The frame of reference (scope) is always global.

4.2 B&R library samples

For a large number of B&R libraries, there are library samples that use either the most important or all function blocks from the corresponding library in an executable task or provide sample code.

This allows you to quickly get an overview of library functionality based on a specific application sample or to reuse parts of the sample code in your own project.

A list of all available library samples can be found in Automation Help. If you know the name of the library you need a sample for, you can also find it directly in Automation Help in the section corresponding to the library.



Overview of all existing library samples:

[Programming \ Examples \ Examples - Libraries](#)

Linking the samples to the FileIO library:

[Programming \ Libraries \ Data access and data storage \ FileIO \ Examples](#)

If executable packages are available for a library, as is the case with FileIO, these can be imported into the project via the Toolbox.

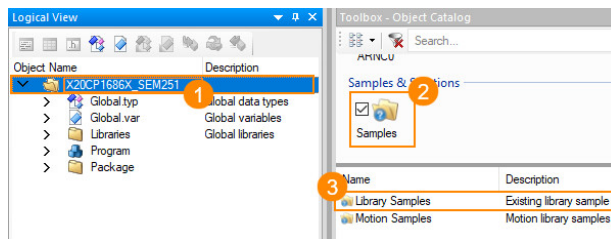


Figure 29: Importing a library sample into the project

- 1) After selecting the entire project or alternatively another package, such as the "package" folder, the option to import a library sample is available in the Toolbox.
- 2) Filtering to "Samples" in the "Samples & Solutions" category makes the search easier.
- 3) Double-click or drag and drop to add a "library sample".

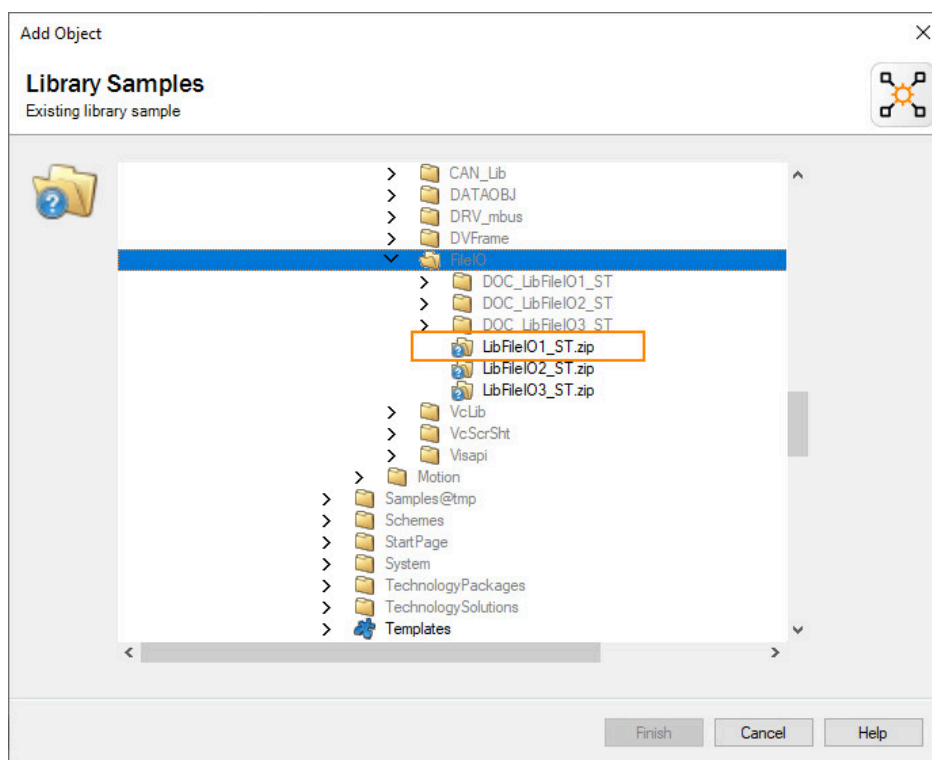


Figure 30: List of available library samples

When Automation Studio is installed, the wizard that opens shows the file structure where the library samples are stored.

All samples are listed in the "Samples\Library" folder.

Here, you can search for the desired library and import an existing sample.

After importing library sample LibFileIO1_ST.zip, the following elements can be found in the project structure:

- 1) The FileIO library itself, as well as the additional library AsBrStr, which is also required in the sample code.
- 2) The folder with the name of library sample "Lib-FileIO1_ST", which contains the executable program.
- 3) The "FileHandling" program itself, which contains all the required variable declarations and the executable code.

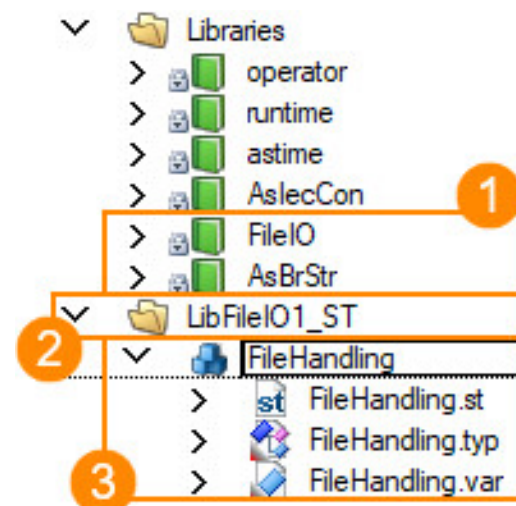


Figure 31: Project structure after importing the library sample

After importing the library sample, the project can be compiled, transferred to the controller and tested.

Library samples often show relatively general use cases that can be adapted to your own specific tasks or at least parts of them can be reused.

Optional task: Communication via TCP / IP

Using the B&R library example LibAsTCP1_ST, two controllers should exchange data with each other via TCP/IP.

Procedure:

1) Define the data

Use data from your control project. Make sure that the structure and data format are identical to the data of your communication partner.

2) Import sample project

Import the example project "LibAsTCP1_ST" in the Logical View.

3) Establish the connection to your communication partner

Choose which station the client and server programs should be loaded to. Alternatively, the client and server applications can be loaded to the same controller or in the simulation environment.

4) Testing communication

Test whether the data is being correctly transferred from the client to the server and vice versa. Use the Watch window for this.

5) Test scenarios in which communication fails

Break the connection to each controller in turn and test whether it can be reestablished by the communication program on its own.

4.3 User libraries

If a suitable library is not available in the list of B&R libraries for an application, it is also possible to create a user library in the project.

Functions, function blocks and type and constant declarations that you have created yourself can then be grouped here. In addition to use in your own project, these user libraries can also be exported, transferred on and therefore used in other projects.

4.3.1 Creating a user library

Similar to adding an existing (B&R) library, the corresponding element from the Toolbox is added to the "Libraries" folder in the Logical View when creating a user library.

In principle, a library can also be added to the project outside of this folder, but care should always be taken to ensure that the project files are clearly structured.

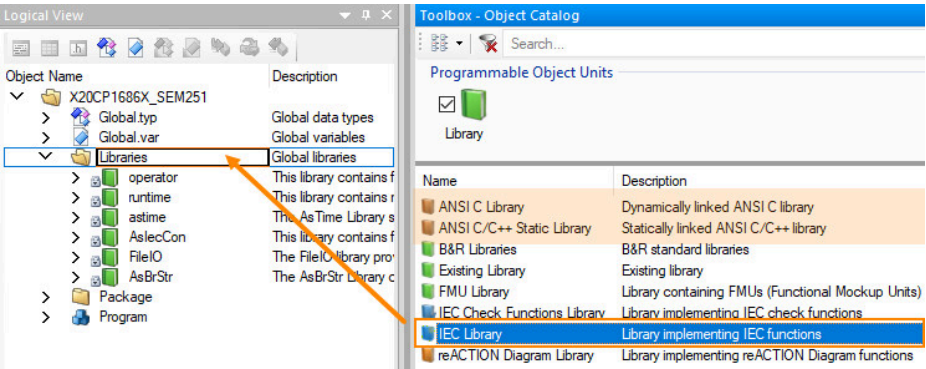


Figure 32: Adding an IEC library

When adding a library, you can choose between ANSI C/C++ libraries and an IEC library.

In the following section, an IEC library should be created. For an explanation of the differences between static and dynamic libraries, see 4.3.3 "Statically and dynamically linked libraries" on page 40.

After import, the library can and should be renamed according to its functionality⁵. In the sample, it is given the name "MyLibrary".

The library initially contains three files:

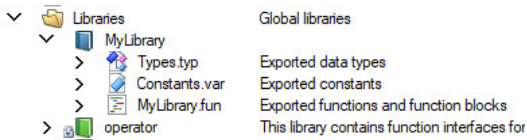


Figure 33: Standard files in an IEC library

- **Types.typ:** Declaration of all user data types that are required by functions and function blocks from MyLibrary⁶
- **Constants.var:** Declaration of required constants
- **MyLibrary.fun:** Declaration file for functions and function blocks

5 L002 in TM232 provides information about naming libraries

6 Other .typ and .var files can also be imported from the Toolbox if this is required for structuring reasons.

Adding a function or function block to a library – Variant 1

The function/function block object can now also be added to the library from the Toolbox via drag and drop.

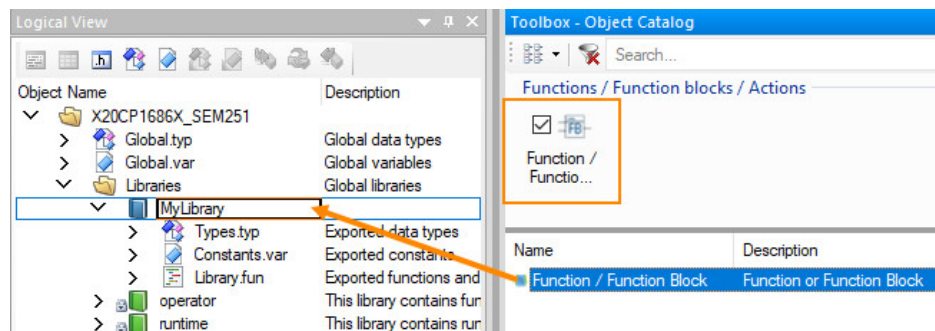
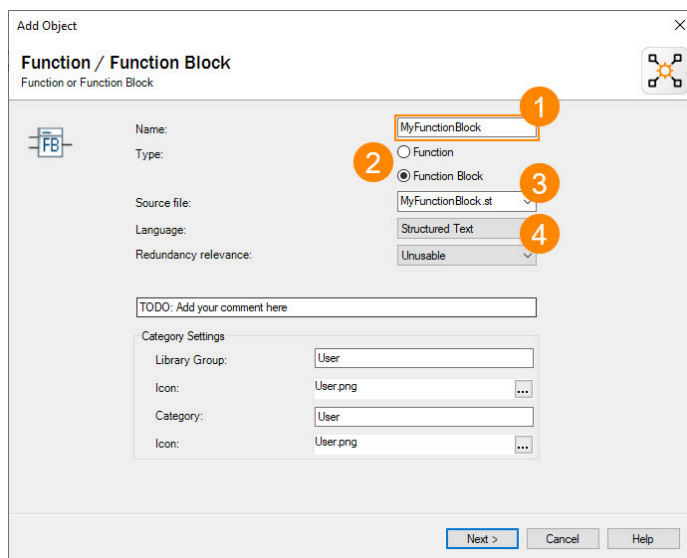


Figure 34: Add function/function block to the library

If this variant is used, a window opens where a few adjustments need to be made.



- 1) Name of the function/function block (will subsequently appear as a declaration in the .fun file).
- 2) Selection of function or function block
- 3) Name of the source code file. By default, this is the name of the block itself with the file extension corresponding to the programming language.
- 4) Desired programming language (an IEC library has been added, so only IEC languages can be used).

Figure 35: Configuration options for functions/function blocks

Both the name of the function block and the name of the source code file can still be changed in the Logical View after creation is complete.

This is particularly useful for text-based programming languages when multiple functions are implemented in the same source code file.

To complete creation of the object, the parameters or interfaces of the function or function block must now be specified.
At least one parameter must be specified; otherwise, creation cannot be completed.

The name, data type and number of parameters can be adjusted in the .fun file at any time after completion. If the declaration is changed, the definition/implementation may also need to be adapted.

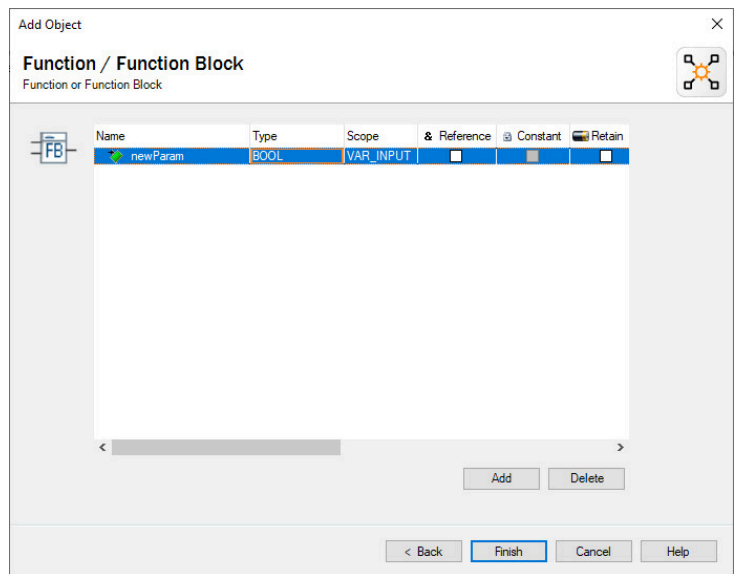


Figure 36: Specifying parameters and interfaces

Adding a function or function block to a library – Variant 2

As an alternative to variant 1, functions or function blocks can be created manually by declaring the desired function or function block in the .fun file for the library as described in 3.2 "Creating your own function blocks" on page 19. A source code file for the preferred programming language is added from the Toolbox, where implementation takes place.



[Programming \ Libraries \ Example: Creating a user library \ Create a C library](#)
[Programming \ Libraries \ Example: Creating a user library \ Create IEC library](#)

Using the functions/function blocks from the library in the project

After successful declaration and implementation, the functions and function blocks can now be used in any program in the project.

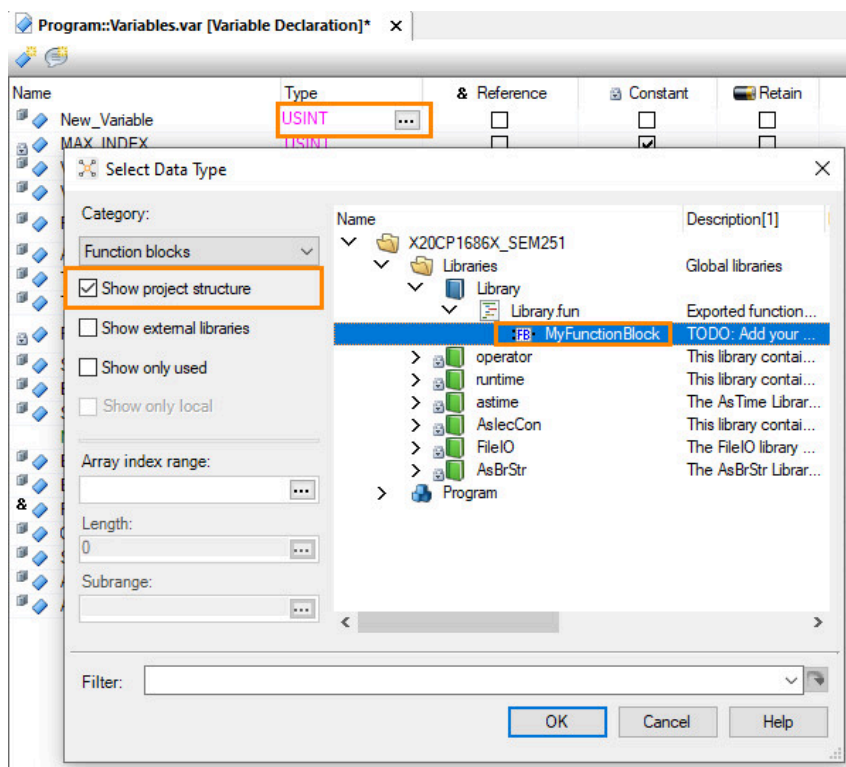


Figure 37: Creating an instance variable for the library block

In the sample, a variable of type `MyFunctionBlock` should be created, which is declared (and implemented) in "Library".

When declaring an instance variable in the variable editor, the corresponding function block is then assigned as usual by accessing the project structure.

Exercise: Creating user library FlexArray

Function `FlexArrayCalcSum()` and function block `FlexArrayInfo()` should be packed into a separate library so that they are globally available in the project and can also be exported.

Procedure:

- 1) Add an empty IEC library from the toolbox and rename it `FlexArray`
- 2) Move the declaration and definition of `FlexArrayCalcSum()` and `FlexArrayInfo()` to the corresponding files in the library.

Any constants used must also be declared within the library.

- 3) Compile your project and test whether the function calls still work.

4.3.2 Exporting and transferring

As already mentioned, the functions, function blocks, data types and constants declared in a library can be used by all programs in the project. If these components are combined in a library, it is also possible to export them together as a library.

Exported libraries can be imported into other projects and used there like standard B&R libraries.

Before export, it is necessary to take existing dependencies in the library into account.

Library dependencies

If a function or a function block in a library accesses a function (or a function block) in another existing library, this is referred to as a **dependency**. This means that the function can only be compiled without errors if the library with functions that were used for implementation is also available in the project.

To export the library you have created yourself, make sure that all dependencies are entered. This is the only way to ensure that the required libraries are also automatically imported when the user library is imported into another project. If dependencies are not taken into account, this leads to error messages referring to unknown functions during the compilation process. It is then necessary to manually import all missing libraries into the project. To avoid this, the library dependencies should always be entered.

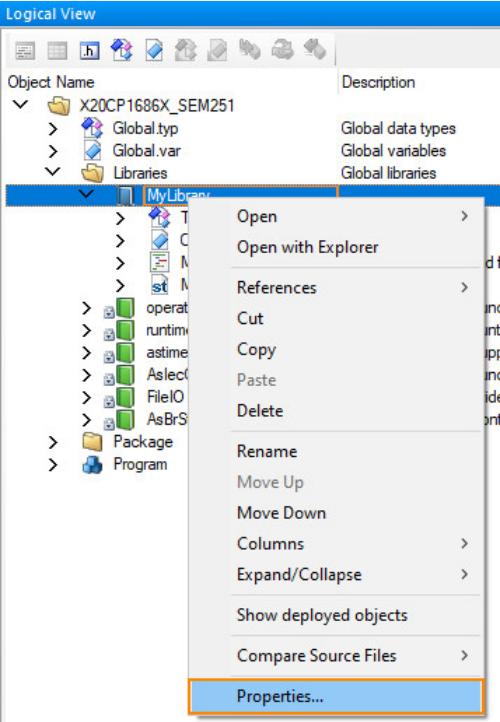


Figure 38: Opening library properties

Right-click on the library and select "Properties" to make various settings.

A properties window with several tabs opens.

A version number can be assigned in the "Details" tab, for example.

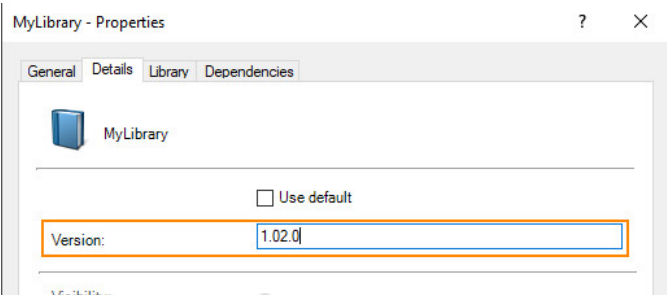
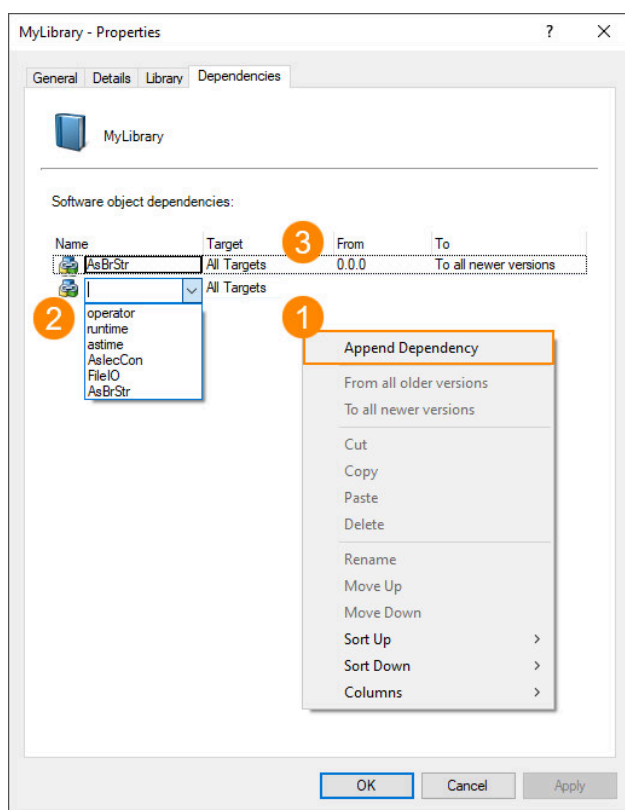


Figure 39: Setting the version number of "MyLibrary"



All libraries used by "MyLibrary" can be entered in the "Dependencies" tab.

- 1) Right-click in the editor area and click "Add dependency".
- 2) In the "Name" field, use the drop-down menu to select the library used by MyLibrary. In the drop-down menu, you can only select from libraries used in the project.
- 3) If the dependency applies only to certain target systems,⁷ this can be limited in the "Target system" column. It is also possible to specify a certain version of a library. This option is useful if a function used is only available starting with a certain version or is no longer available after a certain version. If not specified otherwise, the dependency applies to all target systems and always uses the latest available version of the library to which the dependency exists. With binary libraries, the restriction of dependencies can lead to problems.

Figure 40: Configuring library dependencies

When the library is imported into another project, the libraries in this list are automatically imported if they are not already present there.



[Project management \ Logical View \ Libraries \ Library dependencies](#)
[Programming \ Libraries \ Example: Creating a user library \ Library dependencies](#)

Export

A library can be exported in the **File \ Export library...** menu.

⁷ The target system is not quite correct in this case. This is based on the system generation and the associated processor architecture. SG3 and SGC use a Motorola (BigEndian) architecture and therefore only support certain compiler versions. This can mean that (especially older) libraries can no longer be compiled with current versions.

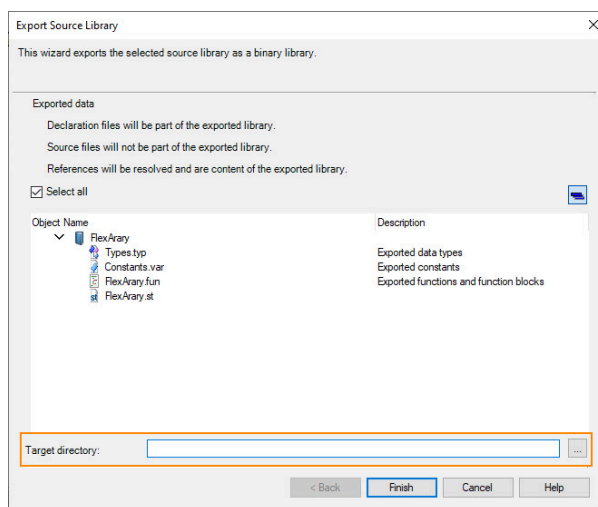


Figure 41: Export options for an IEC library

This exports the FlexArray library as a binary library.

Binary libraries only contain the implementation files in the form of object files, so their content can only be read and used by the compiler. When importing into another project, the files containing the actual implementation of the functions and function blocks can no longer be read by the programmer.

The target directory where the library is saved must also be specified in the export dialog box. All the files that were created during the export and are required for forwarding are found there.



If changes have been made to library properties (version number, dependencies, etc.), the library needs to be compiled before exporting! Otherwise, version conflicts or missing dependencies that cannot be resolved may occur when importing the library into another project.

Task: Exporting the FlexArray library

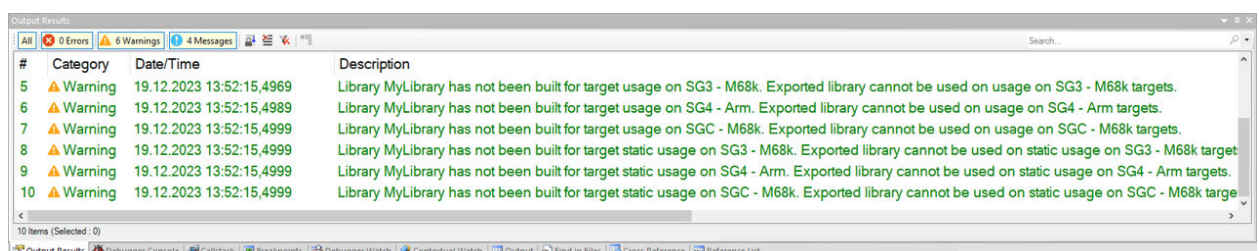
- 1) Declare all required dependencies for your FlexArray user library
- 2) Export your library and import it into another project. Try out different options:
 - New version number
 - Export as static library

Note: After each change to library properties in the project, the project must first be compiled before a new export can successfully take place.

Exporting for different target platforms

If a library is to be created for different system generations (SGC, SG3, SG4) or hardware platforms (IA32, ARM), a corresponding configuration must exist in the project for each desired target platform and processor type and the library must be assigned to each configuration in the software configuration.

A warning is generated in the output window during the export process for each target system for which the library was **not** exported because a corresponding configuration does not exist in the project.

Figure 42: Warning for target systems for which the library was **not** exported

If the library should be exported for multiple target platforms at the same time, this can be done in the Configuration View via batch creation. For detailed instructions, see Automation Help:



[Project management \ Logical View \ Libraries \ Creating a library for different target platforms](#)
[Project management \ Logical View \ Libraries \ Exporting a user library](#)

4.3.3 Statically and dynamically linked libraries

In programming, a distinction is made between statically and dynamically linked libraries. The respective characteristics of the two variants and the associated advantages and disadvantages are presented below.

Statically linked libraries

In a statically bound library, the machine code⁸ is embedded "fixed" (unchangeable) in the machine code of the respective program that uses functions or function blocks during the compilation process.

Advantages

- Changes made to the library after compilation do not jeopardize the ability to execute a program that uses functions from this library. Each program has its own executable copy of the library files made at the time of compilation.
- Faster processing at runtime because loading "external machine code" is not necessary. This advantage is particularly important when calling up a large number of library functions.

Disadvantages

- Each program has its own "local" copy (in machine code) of a function, even if different programs use the same function.
- More memory required for executable files because each program has its own "local" copy of the function's machine code.
- Changes to a function in the library require all programs that use the function to be recompiled.

Dynamically linked libraries

With a dynamically linked library, a separate local machine code copy of the function is not created for each individual program during the compilation process. At runtime, the same library function machine code is accessed by each program.

Dynamic libraries and their functions exist "outside" the machine code of the programs that access them.

⁸ Binary, executable code in memory that is generated by the compiler from the human-readable source code and can be executed by the processor.

Advantages

- When compiling the project, only one copy of the library (or the library functions used) is created (outside of all programs). This results in lower memory requirements for machine code, especially if many programs access the same functions.
- Different programs access the same library function machine code, so each program does not need its own copy.
- A dynamic library can be modified without having to make changes to (and recompile) the programs that use the library's functions.

Disadvantages

- Longer program runtimes because memory areas outside the program's machine code need to be accessed at runtime when a library function is called.
- Changes to a dynamic library can result in programs that use the library no longer working due to incompatibilities⁹.

Static and dynamic libraries in Automation Studio

For more detailed information about selection criteria and the files generated during export for the different variants, see Automation Help:



[Programming \ Libraries \ Statically linked libraries](#)

[Programming \ Libraries \ Dynamically linked libraries](#)

When creating a C/C++ user library in Automation Studio, it is necessary to decide when importing the empty library into the project if it should later be linked dynamically or statically.

In the case of an IEC library, it is initially always a dynamically linked library.

This can be adjusted later in the software configuration for the controller, however, and the library can be changed to static.

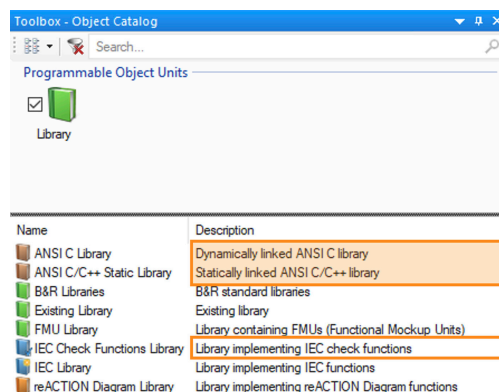


Figure 43: Choosing between a static and dynamic library

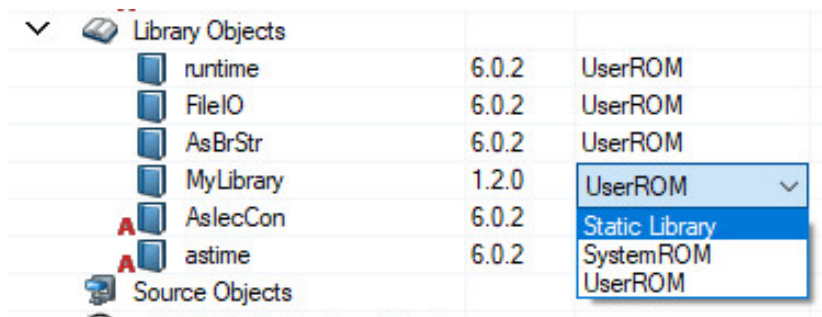


Figure 44: Configuring an IEC library as static

An IEC user library created in the project can be configured as a static library in the software configuration for the controller if the entry "Static library" is selected instead of "UserROM".

IEC libraries are generally not used as static libraries.

If you want to keep the option to choose between a static or dynamic IEC library after exporting the library, the following steps must be carried out:

⁹ This disadvantage does not occur in AS projects because changes to all global information in the project (to a library, for example) always result in the entire project having to be recompiled. If incompatibilities occur, the project cannot be transferred to the target system.

- 1) Configure the library as static in the project.
- 2) Compile the project.
- 3) Export the library.

This library can then be used in any other project either as a static or dynamic library.

5 mapp Technology

With **mapp Technology**, B&R offers an extension of the concept of functions, function blocks and libraries.

mapp stands for **modular application**. Various Technology Packages provide a wide range of tools for efficient and fast configuration and programming of complex applications.



5.1 Concept

B&R currently offers seven different mapp Technology Packages.



mapp
Services

mapp Services can be used to configure the entire data infrastructure of a machine. These include alarm handling, data management, notification systems, recipe management, energy management and user management.



mapp
View

mapp View can be used to quickly and easily create powerful HMI applications. Integrating the user role system also allows customized functionality.



mapp
Motion

mapp Motion makes it possible for drives to be easily commissioned and for single-axis and multi-axis applications to be programmed with reduced effort.



mapp
Control

mapp Control allows various control processes to be quickly implemented and integrated in a project quickly and without major programming effort.



mapp
Vision

mapp Vision provides a quick and easy way to implement camera system image acquisition and illumination.



mapp
Safety

mapp Safety allows safety applications to be created efficiently and reliably in a machine.



mapp
Cockpit

mapp Cockpit provides extensive options for quick and easy diagnostics and configuration.



[mapp Technology](#)

Each Technology Package is divided into different application areas.

By importing a Technology Package, ready-made solutions and standard applications are installed, enabling quick and easy commissioning and programming.

In the case of mapp Motion, there are four application areas:

- mapp Axis
- mapp CNC
- mapp Robotics
- mapp Trak

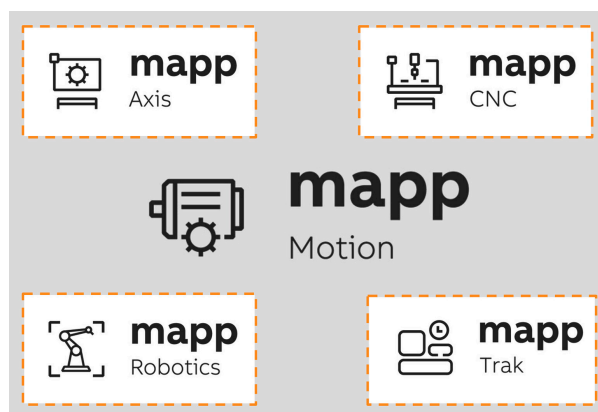


Figure 45: The four application areas included in the mapp Motion Technology Package

5.2 Components of a mapp Technology Package

A mapp Technology Package (such as mapp Motion) can be thought of as a package of these components:

mapp-Technology Package

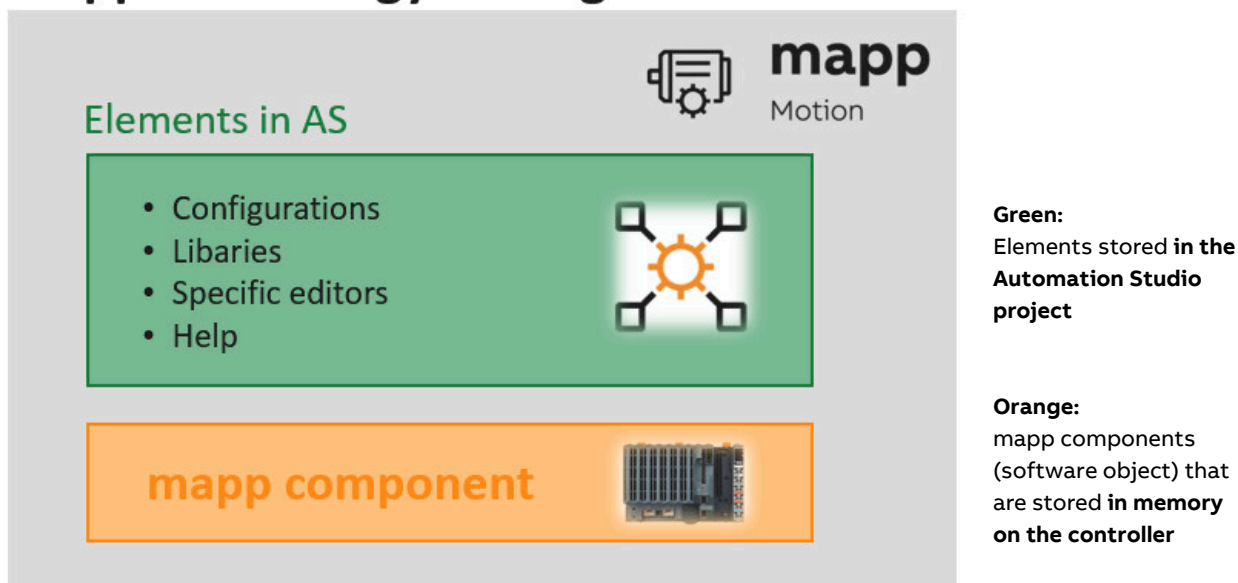


Figure 46: Components of a mapp Technology Package

5.2.1 mapp components

The mapp components are key elements of mapp Technology. A mapp component can be thought of as a software object in the controller's memory. The information stored in this software object specifies all the settings required to determine the behavior of an axis, for example.

A mapp component is configured quickly and easily in Automation Studio via the corresponding configuration file. The interfaces of the mapp components are also standardized according to IEC, allowing access with function blocks.

A mapp component is uniquely identified among all existing mapp components via a mapp Link (MpLink). The mapp Link is used to access the component via software (functions and function blocks) or if data should be exchanged or transferred from mapp components in the mapp framework ([5.3 "mapp framework" on page 49](#)).

5.2.2 Components in Automation Studio

After importing a mapp Technology Package, elements are available that allow the corresponding technology to be implemented for a specific application quickly and easily.

Configurations

mapp components are configured in their respective configuration file. The motto is "**Configuring instead of programming**". The mapp component is created as a data object on the target system based on the settings made in the configuration file.

Function blocks (such as `MpAxisBasicConfig`) can be used to make changes to the configuration file. The changes to the target system are then only overwritten again during a transfer. It is possible to open a view where the differences between the project and the target system are listed (Online -> Comparison -> Automation Components).

Libraries

The libraries belonging to a mapp Technology Package contain functions and function blocks that enable software access to the mapp components in the Technology Package.

The functions and function blocks conform to (IEC, PLC open) standards and are therefore compatible with all software solutions that are also based on these standards.

Automation Help documentation

When a Technology Package is installed, the corresponding information is also automatically added to or updated in Automation Help. Information is always available for the newest and latest installed Technology Package.

Editor

Some Technology Packages, such as mapp View, come with their own editor (e.g. for web-based HMI applications). It is available after installing the Technology Package in Automation Studio and is another helpful tool for creating the required application.

5.2.3 Use in Automation Studio

When installing a Technology Package, all package components are automatically integrated into Automation Studio and can then be used in the project.

Installing a mapp Technology Package

Technology Packages are installed via "Extras \ Upgrades..".

- 1) Select the location from which the Technology Package is downloaded/installed:
 - B&R website: Internet connection required
 - Local: Installation file in the local directory
- 2) Use the "Technology Packages" filter to narrow down the search results in the categories.
- 3) The latest available version of all packages is always displayed by default. If an older version is required (e.g. for compatibility reasons), this can be specified in the filter field.

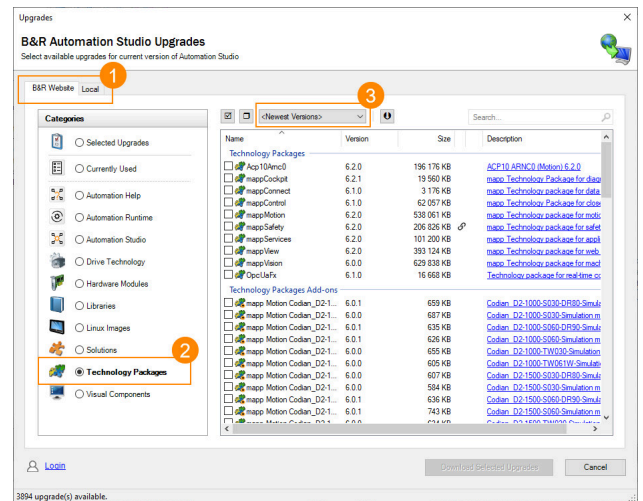


Figure 47: Downloading mapp Technology Packages

Setting the version of mapp Technology Packages in the project

All installed mapp versions are stored in Automation Studio. The respective package version is set for the entire project via "Project \ Change runtime versions..".

- 1) You can choose between the installed versions in the drop-down menu of each mapp category.
- 2) "Not defined" means that the technology package is not used in the project.

Some technology packages are dependent on other technology packages, which must then also be used in the project. For example, you also need mapp View when using mapp Cockpit. Information about this can be found in the help section of the respective technology package.

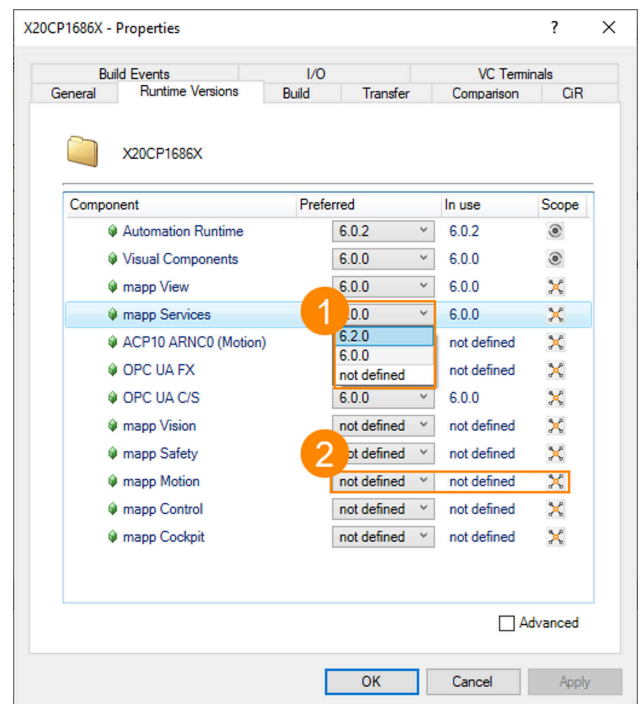


Figure 48: Setting mapp versions in a project

The corresponding Automation Help pages are only available once a mapp Technology Package has been installed. The elements from the Toolbox belonging to the mapp Technology Package can also only be added to the project from this point onwards.

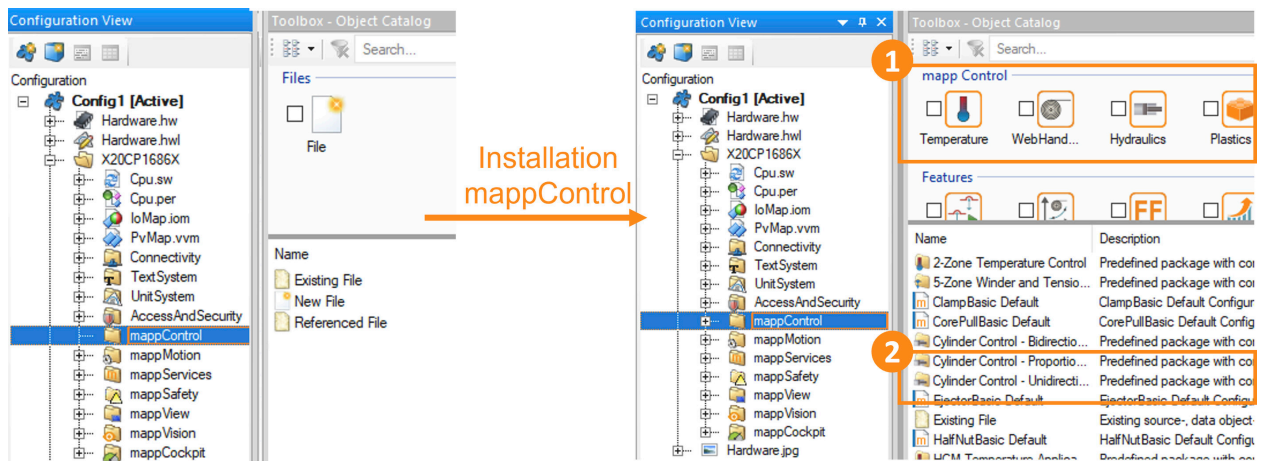
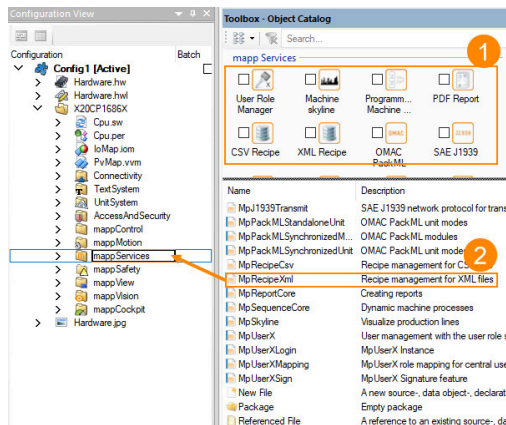


Figure 49: Left: No configuration files available for mapp Control, Right: Configuration files available in the Toolbox

If the corresponding mapp technology package has been installed in Automation Studio, the configuration files of the technology package are displayed in the toolbox.

- 1) The filters for the individual mapp Services components (such as XML Recipe) are used to limit the search results.
- 2) From the search results, the configurations can be placed in the previously selected folder in the Configuration View using drag-and-drop or by double-clicking¹⁰.



Using mapp Technology Package components

To use mapp components, the first step is to add a configuration file to the Configuration View. In the example shown, two configuration files (AlarmX.mpalarmxcore and RecipeXml.mprecipexml) from the mapp Services technology package were added to the Configuration View.

¹⁰ The mapp configuration files must be stored in the corresponding folders. It is not permitted to store a mapp Control component with mapp Services, for example. Using the filter selection prevents this.

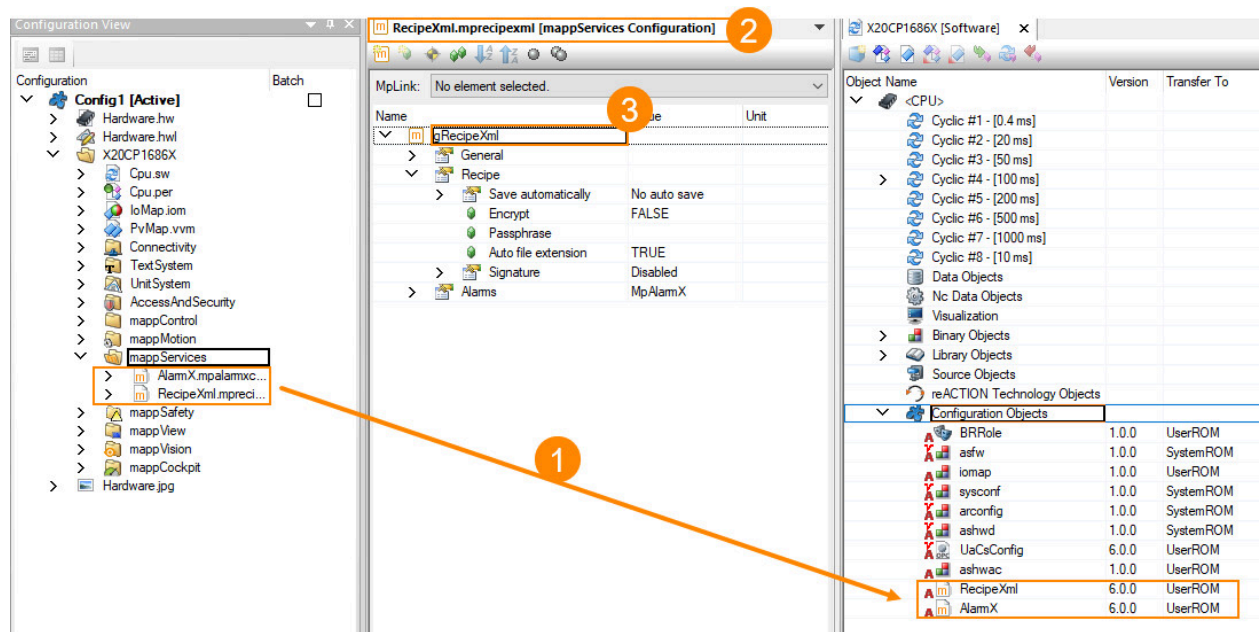


Figure 50: Relationship between mapp components and configuration files

- 1) The configuration files added in the Configuration View (AlarmX and RecipeXml in the example) can be found with the configuration objects for the respective software after compilation. After transferring the project to the controller, this becomes the software object for the mapp component in the controller's memory.
- 2) Double-click on the file added in the Configuration View to open the configuration file for the corresponding mapp component. Default values are already entered here, which can then be adapted to your own requirements.
- 3) The name of the mapp component (gRecipeXml in the sample) can be modified if necessary. This name is used if the mapp Link is required, whereby the mapp Link is the address of the software object. The mapp Link for mapp component RecipeXml is thus transferred to any function block with ADR (gRecipeXml).

Optional task: Saving and loading a recipe

Using the mapp Services technology package, mappRecipe should be used to store the values of a `PizzaType` variable in an XML file and read them back from it.

Procedure

- 1) Configuring the file device

Create a file device with the name 'USER' and the path "USER_PATH" in the CPU configuration under "File devices"

- 2) Work through the Getting Started tutorial from the help documentation for creating a recipe management to implement saving and loading a variable of the type `PizzaType`.



[Services \ mapp Services \ mapp Recipe: Recipe management \ Getting started \ Creating a recipe management system](#)

- 3) Use the Watch window to operate inputs `Save` and `Load` of your `MpRecipeXml` function block and test your recipe management.

5.3 mapp framework

mapp Technology allows specific technologies to be quickly and easily put to use in a larger overall system and the individual mapp components can also exchange data and information with each other, which are great advantages.

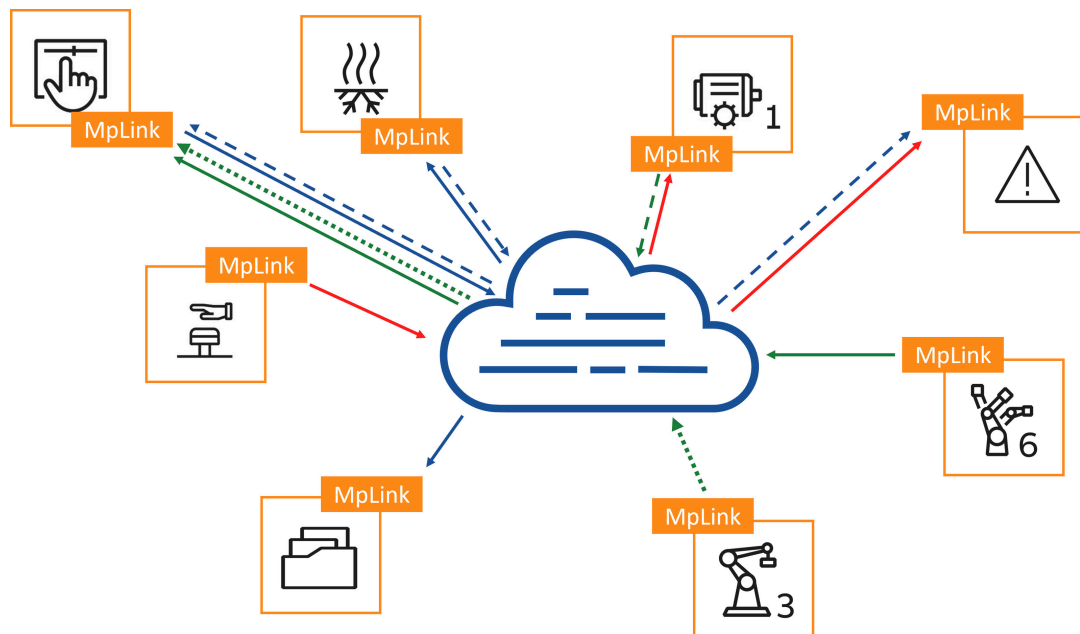


Figure 51: Communication between individual mapp components via the mapp framework

This is known as the mapp framework, where the mapp components (on the controller) are integrated and can be clearly identified by their respective mapp Link. The mapp framework described here is an "ecosystem" in which the individual mapp components exchange information with each other¹¹.

Each mapp component in the mapp framework can act as a data source (e.g. sending energy consumption or alarm messages) and as a data sink (e.g. retrieving alarm messages for display in an HMI application).

The **MpBase** library is required to exchange information between mapp components via the mapp framework, and this library is automatically imported into the project when the first configuration file for any mapp component is added.



Each mapp component fits seamlessly into the complex mapp framework. No further actions by the programmer are required for this!



[mapp Technology \ Concept \ Communication between mapp packages](#)

¹¹ This should not be confused with mapp Framework as a product, which is an Automation Studio project / Automation Studio plug-in that should make it easier to get started with mapp Technology and [can be downloaded from the website](#).

6 Summary

With Automation Studio, B&R provides a wide range of standard libraries for frequently used applications.

In addition, programming sophisticated and complex mechatronic solutions, modern HMI applications, data exchange methods, etc., is made much easier by the mapp Technology Packages developed by B&R.

For more specific applications, functions and function blocks can also be created in various programming languages. Automation Studio provides support for exporting and transferring user libraries.

When using your own functions, function blocks and libraries, we recommended that you use standardized interfaces and behavior for reasons of maintainability.

Before exporting, it is necessary to ensure that dependencies have been entered. Documenting a user library is always advisable because this ensures that it can be used efficiently and without errors.

7 Solutions

Solution: WritePizzaString()

The return value of a function cannot be a STRING, ARRAY or STRUCT, so the `WritePizzaString()` function needs write access to a string that exists "outside" the function. In the proposed solution, this is implemented by transferring the target string as `VAR_IN_OUT`.

`fSTRING_LENGTH` is a constant of the type `DINT`.

Function declarations (.fun file)

```
FUNCTION WritePizzaString : DINT
    VAR_INPUT
        Pizza : Pizza4Type;
    END_VAR
    VAR_IN_OUT
        DestString : STRING[fSTRING_LENGTH];
    END_VAR
    VAR
        NumAsStr : STRING[fSTRING_LENGTH];
        TempStr : STRING[fSTRING_LENGTH];
    END_VAR
END_FUNCTION
```

Function definitions (.st file)

```
FUNCTION WritePizzaString

    // Price
    DestString := 'Price:=';
    NumAsStr := REAL_TO_STRING(Pizza.Price.Euro);
    IF brsstrlen(ADR(NumAsStr)) < (SIZEOF(DestString) - brsstrlen(ADR(DestString))) THEN
        brsstrcat(ADR(DestString), ADR(NumAsStr));
    ELSE
        DestString := '';
        WritePizzaString_AsBrStr := fERR_DEST_STRING_TO_SHORT;
        RETURN;
    END_IF;
    // Size
    TempStr := ';Size:=';
    IF brsstrlen(ADR(TempStr)) < (SIZEOF(DestString) - brsstrlen(ADR(DestString))) THEN
        brsstrcat(ADR(DestString), ADR(TempStr));
    ELSE
        DestString := '';
        WritePizzaString_AsBrStr := fERR_DEST_STRING_TO_SHORT;
        RETURN;
    END_IF;
    NumAsStr := REAL_TO_STRING(Pizza.Size);
    IF brsstrlen(ADR(NumAsStr)) < (SIZEOF(DestString) - brsstrlen(ADR(DestString))) THEN
        brsstrcat(ADR(DestString), ADR(NumAsStr));
    ELSE
        WritePizzaString_AsBrStr := fERR_DEST_STRING_TO_SHORT;
        RETURN;
    END_IF;
    // LeaveCheese
    TempStr := ';LeaveCheese:=';
    IF brsstrlen(ADR(TempStr)) < (SIZEOF(DestString) - brsstrlen(ADR(DestString))) THEN
        brsstrcat(ADR(DestString), ADR(TempStr));
    ELSE
        DestString := '';
        WritePizzaString_AsBrStr := fERR_DEST_STRING_TO_SHORT;
        RETURN;
    END_IF;
```

```
END_IF;
IF Pizza.LeaveCheese THEN
    NumAsStr := 'TRUE';
ELSE
    NumAsStr := 'FALSE';
END_IF;
IF brsstrlen(ADR(NumAsStr)) < (SIZEOF(DestString) - brsstrlen(ADR(DestString))) THEN
    brsstrcat(ADR(DestString), ADR(NumAsStr));
ELSE
    DestString := '';
    WritePizzaString_AsBrStr := fERR_DEST_STRING_TO_SHORT;
    RETURN;
END_IF;
// Last semicolon
NumAsStr := ';';
IF brsstrlen(ADR(NumAsStr)) < (SIZEOF(DestString) - brsstrlen(ADR(DestString))) THEN
    brsstrcat(ADR(DestString), ADR(NumAsStr));
ELSE
    DestString := '';
    WritePizzaString_AsBrStr := fERR_DEST_STRING_TO_SHORT;
    RETURN;
END_IF;
// All good
WritePizzaString_AsBrStr := fERR_OK;

END_FUNCTION
```

Use of WritePizzaString()

Declaration	<pre>VAR Pizza : PizzaType; WritePizzaStringResult : STRING[fSTRING_LENGTH]; WritePizzaStringStatus : DINT; END_VAR VAR CONSTANT fSTRING_LENGTH : DINT := 100; END_VAR</pre>
Program code	<pre>PROGRAM _CYCLIC WritePizzaStringStatus := WritePizzaString(Pizza, WritePizzaStringResult); END_PROGRAM</pre>

Table 1: Use of WritePizzaString()

Solution FlexArrayCalcSum()

Function declaration (.fun file)

```
FUNCTION FlexArrayCalcSum : REAL
    VAR_INPUT
        StartOfArray : REFERENCE TO REAL;
        NumArrayElements : UDINT;
    END_VAR
    VAR
        i : UDINT;
        Sum : REAL;
        ActArrayElement : REFERENCE TO REAL;
    END_VAR
END_FUNCTION
```

Function definition (.st file)

```
FUNCTION FlexArrayCalcSum

    IF NumArrayElements = 0 OR ADR(StartOfArray) = 0 THEN
        Sum := 0.0;
    ELSE
        FOR i := 0 TO (NumArrayElements - 1) DO
            ActArrayElement ACCESS (ADR(StartOfArray) + (i * SIZEOF(StartOfArray)));
            Sum := Sum + ActArrayElement;
        END_FOR;
    END_IF;
    FlexArrayCalcSum := Sum;

END_FUNCTION
```

Using FlexArrayCalcSum()

Declaration	<pre>VAR ValueFlexArray : ARRAY[0..9] OF REAL; SumOfFlexArray : LREAL; END_VAR</pre>
Program code	<pre>PROGRAM _CYCLIC SumOfFlexArray := FlexArrayCalcSum(ADR(ValueFlexArray), SIZEOF(ValueFlexArray) / SIZEOF(ValueFlexArray[0])); END_PROGRAM</pre>

Solution FlexArrayInfo()

Function block declaration (.fun file)

```
FUNCTION_BLOCK FlexArrayInfo
    VAR_INPUT
        StartOfArray : REFERENCE TO REAL;
        NumArrayElements : UDINT;
    END_VAR
    VAR_OUTPUT
        Sum : REAL;
        Average : REAL;
        MinValue : REAL;
        MaxValue : REAL;
    END_VAR
    VAR
        i : UDINT;
        ActArrayElement : REFERENCE TO REAL;
    END_VAR
END_FUNCTION_BLOCK
```

Function block definition (.st file)

```
FUNCTION_BLOCK FlexArrayInfo

    IF NumArrayElements = 0 OR ADR(StartOfArray) = 0 THEN
        Sum := 0;
        MinValue := 0;
        MaxValue := 0;
        Average := 0;
```

```

ELSE
    // Calculate values
    Sum := FlexArrayCalcSum(ADR(StartOfArray), NumArrayElements);
    MinValue := StartOfArray;
    MaxValue := StartOfArray;
    FOR i := 0 TO (NumArrayElements - 1) DO
        ActArrayElement ACCESS (ADR(StartOfArray) + (i * SIZEOF(StartOfArray)));
        MinValue := MIN(MinValue, ActArrayElement);
        MaxValue := MAX(MaxValue, ActArrayElement);
    END_FOR;
    Average := Sum / UDINT_TO_REAL(NumArrayElements);
END_IF;

END_FUNCTION_BLOCK

```

Using the block

Declaration	<pre> VAR FlexArrayInfo_0 : FlexArrayInfo; ValueArray : ARRAY[0..19] OF REAL; END_VAR </pre>
Program code	<pre> PROGRAM _CYCLIC FlexArrayInfo_0.StartOfArray := ADR(ValueArray); FlexArrayInfo_0.NumArrayElements := SIZEOF(ValueArray) / SIZEOF(ValueArray[0]); FlexArrayInfo_0(); END_PROGRAM </pre>

Standardized interfaces for function blocks

Two variants for implementing standardized interfaces for `FlexArrayInfo` are shown.

Variant 1 - fast

The values for all outputs are calculated within one cycle, making this block "faster".

Variant 2 - slow

This variant works internally with a state machine and therefore requires more than just one cycle to complete the calculations.

The advantage of this variant is that the source code remains clear even for more extensive function block tasks.

Declaration of the block (variant 1)

```
FUNCTION_BLOCK FlexArrayInfoFast
VAR_INPUT
    Execute : BOOL;
    StartOfArray : REFERENCE TO REAL;
    NumArrayElements : UDINT;
END_VAR
VAR_OUTPUT
    Busy : BOOL;
    Done : BOOL;
    Error : BOOL;
    StatusID : DINT;
    Sum : REAL;
    Average : REAL;
    MinValue : REAL;
    MaxValue : REAL;
END_VAR
VAR
    i : UDINT;
    ActArrayElement : REFERENCE TO REAL;
    zzEdge00000 : BOOL;
END_VAR
END_FUNCTION_BLOCK
```

Definition of the block (variant 1)

```
// Single Cycle Execute
FUNCTION_BLOCK FlexArrayInfoFast

IF EDGEPOS(Execute) THEN
    IF NumArrayElements = 0 THEN
        // Set error status
        Error := TRUE;
        StatusID := fERR_NUM_ELEMENTS_ZERO;
    ELSIF ADR(StartOfArray) = 0 THEN
        // Set error status
        Error := TRUE;
        StatusID := fERR_NULL_POINTER;
    ELSE
        // Calculate values
        Sum := FlexArrayCalcSum(ADR(StartOfArray), NumArrayElements);
        MinValue := StartOfArray;
        MaxValue := StartOfArray;
        FOR i := 0 TO (NumArrayElements - 1) DO
            ActArrayElement ACCESS (ADR(StartOfArray) + (i * SIZEOF(StartOfArray)));
            MinValue := MIN(MinValue, ActArrayElement);
            MaxValue := MAX(MaxValue, ActArrayElement);
            IF ActArrayElement < 0.0 THEN
                StatusID := fWRN_NEGATIVE_VALUES;
            END_IF;
        END_FOR;
        Average := Sum / UDINT_TO_REAL(NumArrayElements);
        // Set done status
        Done := TRUE;
```

```

        END_IF;
    ELSIF Execute THEN
        // Execute is still set -> keep output state
    ELSE
        // Reset values
        Sum := 0.0;
        MinValue := 0.0;
        MaxValue := 0.0;
        Average := 0.0;
        // Reset status
        Busy := FALSE;
        Done := FALSE;
        Error := FALSE;
        StatusID := ERR_OK;
    END_IF;
END_FUNCTION_BLOCK

```

Declaration of the block (variant 2)

```

FUNCTION_BLOCK FlexArrayInfoSlow
    VAR_INPUT
        Execute : BOOL;
        StartOfArray : REFERENCE TO REAL;
        NumArrayElements : UDINT;
    END_VAR
    VAR_OUTPUT
        Busy : BOOL;
        Done : BOOL;
        Error : BOOL;
        StatusID : DINT;
        Sum : REAL;
        Average : REAL;
        MinValue : REAL;
        MaxValue : REAL;
    END_VAR
    VAR
        i : UDINT;
        ActArrayElement : REFERENCE TO REAL;
        State : FlexArrayInfoStateEnum;
    END_VAR
END_FUNCTION_BLOCK

```

Definition of the block (variant 2)

```

// State Machine for Execute / Done FB
FUNCTION_BLOCK FlexArrayInfoSlow

    CASE State OF
        ST_IDLE:
            IF Execute THEN
                Busy := TRUE;
                State := ST_CHECK_PARAMETERS;
            END_IF;

        ST_CHECK_PARAMETERS:
            IF NumArrayElements = 0 THEN
                Busy := FALSE;
                StatusID := fERR_NUM_ELEMENTS_ZERO;
                State := ST_ERROR;
            ELSIF ADR(StartOfArray) = 0 THEN
                Busy := FALSE;
                StatusID := fERR_NULL_POINTER;
                State := ST_ERROR;
            ELSE

```

```

        // All good -> continue
        State := ST_CALCULATE;
    END_IF;

ST_CALCULATE:
    // Calculate values
    Sum := FlexArrayCalcSum(ADR(StartOfArray), NumArrayElements);
    MinValue := StartOfArray;
    MaxValue := StartOfArray;
    FOR i := 0 TO (NumArrayElements - 1) DO
        ActArrayElement ACCESS (ADR(StartOfArray) + (i * SIZEOF(StartOfArray)));
        MinValue := MIN(MinValue, ActArrayElement);
        MaxValue := MAX(MaxValue, ActArrayElement);
        IF ActArrayElement < 0.0 THEN
            StatusID := fWRN_NEGATIVE_VALUES;
        END_IF;
    END_FOR;
    Average := Sum / UDINT_TO_REAL(NumArrayElements);
    State := ST_DONE;

ST_DONE:
    // Set done and change state
    Busy := FALSE;
    Done := TRUE;
    IF NOT Execute THEN
        State := ST_RESET;
    END_IF;

ST_ERROR:
    // Set Error
    Busy := FALSE;
    Error := TRUE;
    IF NOT Execute THEN
        State := ST_RESET;
    END_IF;

ST_RESET:
    // Reset values
    Sum := 0.0;
    MinValue := 0.0;
    MaxValue := 0.0;
    Average := 0.0;
    // Reset status
    Error := FALSE;
    Done := FALSE;
    State := ST_IDLE;

END_CASE;

```

In this case, an enumeration for state variable State must also be declared in the program's .typ file:

```

TYPE
    FlexArrayInfoStateEnum :
    (
        ST_IDLE,
        ST_CHECK_PARAMETERS,
        ST_CALCULATE,
        ST_DONE,
        ST_ERROR,
        ST_RESET
    );
END_TYPE

```




B&R
Industrial Automation GmbH
A member of the ABB Group
B&R Straße 1
5142 Eggelsberg, Austria
office@br-automation.com

t +43 7748 6586-0
f +43 7748 6586-26

br-automation.com

